arXiv:2511.11292v1 [cs.PL] 14 Nov 2025

The Jasmin Compiler Preserves Cryptographic Security

Formalizing Cryptographic Security with ITrees and Proving Compiler Preservation with RHL

SANTIAGO ARRANZ-OLMOS, MPI-SP, Germany
GILLES BARTHE, MPI-SP, Germany and IMDEA Software Institute, Spain
LIONEL BLATTER, MPI-SP, Germany
BENJAMIN GRÉGOIRE, Inria, France
VINCENT LAPORTE, Inria, France and Université de Lorraine, France
PAOLO TORRINI, Inria, France

Jasmin is a programming and verification framework for developing efficient, formally verified, cryptographic implementations. A main component of the framework is the Jasmin compiler, which empowers programmers to write efficient implementations of state-of-the-art cryptographic primitives, including post-quantum cryptographic standards. The Jasmin compiler is proven functionally correct in the Rocq prover. However, this functional correctness statement does not apply to nonterminating or probabilistic computations, which are essential features in cryptography.

In this paper, we significantly enhance the guarantees of the compiler by showing, in the Rocq prover, that its front-end (25 out of 30 passes) preserves cryptographic security. To this end, we first define a Relational Hoare Logic tailored for compiler correctness proofs. We prove the soundness of our logic w.r.t. a new denotational semantics of Jasmin programs based on interaction trees. Secondly, we use our program logic to prove the functional correctness of the (unmodified) Jasmin compiler w.r.t. said semantics. Lastly, we formalize cryptographic security—focusing on IND-CCA—with interaction trees and prove that the Jasmin compiler preserves cryptographic security.

1 Introduction

Security proofs are at the heart of modern cryptography: (almost) every new cryptographic scheme comes with a mathematical proof of security. However, these cryptographic schemes often eschew security critical components to focus on idealized schemes, sometimes known as cryptographic cores. The emphasis on cryptographic cores is primarily justified by the need to ensure that the overall analysis remains tractable with pen-and-paper proofs. However, this emphasis leads to dangerous gaps between security proofs and cryptographic schemes as well as between cryptographic schemes and their implementations. Both gaps can (and have been) exploited to launch devastating attacks on deployed cryptographic implementations [27].

Computer-aided cryptography [14] aims to address these gaps by developing and applying methods and tools for reasoning about cryptographic implementations, rather than cryptographic cores. To date, computer-aided cryptography has been used successfully to deploy high-assurance code in mainstream cryptographic libraries and applications. While these deployments avoid the pitfalls of reasoning on cryptographic cores, they still rely on the assumption that compilers preserve cryptographic security: (almost) all verifications are performed at source-level implementations, and implicitly assume that the security guarantees hold for assembly-level implementations. While reasonable, the assumption has never been established formally.

This paper addresses this gap by proving that the Jasmin [5, 7] compiler front-end preserves cryptographic security. The proof is mechanized in the Rocq prover [59], and provided as an artifact. Our choice of the Jasmin compiler is motivated by two reasons. First, Jasmin is a state-of-the-art

Authors' Contact Information: Santiago Arranz-Olmos, MPI-SP, Bochum, Germany, santiago.arranz-olmos@mpi-sp.org; Gilles Barthe, MPI-SP, Bochum, Germany and IMDEA Software Institute, Madrid, Spain, gilles.barthe@mpi-sp.org; Lionel Blatter, MPI-SP, Bochum, Germany, lionel.blatter@mpi-sp.org; Benjamin Grégoire, Inria, Sophia Antipolis, France, benjamin.gregoire@inria.fr; Vincent Laporte, Inria, Nancy, France and Université de Lorraine, Nancy, France, Vincent.Laporte@inria.fr; Paolo Torrini, Inria, Sophia Antipolis, France, paolo.torrini@inria.fr.

framework for high-assurance high-speed cryptography. It has been used to write and verify many cryptographic implementations, and in particular a highly optimized, vectorized implementation of the post-quantum key encapsulation mechanism ML-KEM [50], which was recently standardized by the National Institute of Standards and Technology (NIST)—a KEM is a special form of public-key encryption that is used by two parties to share a secret key. Secondly, the Jasmin compiler is formally specified and verified in the Rocq prover. This formalization provides an excellent starting point for our work.

Proving that a compiler preserves cryptographic security requires two ingredients that have not been considered so far in the context of the Jasmin compiler, namely:

- *Probabilistic Semantics.* Randomness is ubiquitous in cryptography; therefore, one must consider randomized programs, and equip them with a probabilistic semantics. Furthermore, many cryptographic implementations, and in particular post-quantum implementations, only achieve a probabilistic notion of termination, known as almost sure termination—meaning that the probability that execution diverges is zero. Therefore, one must extend the usual compiler correctness statements to account for such probabilistic behaviors.
- *Security Experiments.* Most notions of cryptographic security are defined by security experiments, in which an adversary interacts with a challenger with access to the cryptographic system. Every security experiment also defines winning conditions, which determine when the adversary successfully broke the cryptographic system. Therefore, one must extend the usual compiler correctness statements to reason about winning probabilities in an interactive setting.

We extend the Jasmin framework to support both ingredients, formalize them, and use the Rocq prover to show that the Jasmin compiler preserves cryptographic security. We elaborate on our main contributions below.

Contribution 1: Relational Hoare Logic for Compiler Correctness. In Section 4, we define two Relational Hoare Logics (RHLs) for Jasmin programs. Our first logic is a core RHL akin to [24]. Our second logic elaborates on the first to be tailored for compiler correctness proofs: for instance, we update the notion of validity and provide specialized rules for common compiler patterns. This second logic forms the basis of our new proof of the Jasmin compiler. Both logics are proved sound w.r.t. a denotational semantics of programs. Concretely, our denotational semantics is based on the general framework of interaction trees [62]. However, many of the ideas of our tailored RHL are independent of interaction trees and of Jasmin.

Contribution 2: New Semantics and New Proofs for the Old Jasmin Compiler. We leverage the framework of interaction trees to define a new semantics of Jasmin programs that accounts for infinite and probabilistic executions. Then, in Section 5, we use our Relational Hoare Logic to lift the compiler correctness theorem of Jasmin to this new semantics. Critically, we *do not modify* the Jasmin compiler, and we largely *reuse* existing correctness proofs. Pleasingly, we employ the same generic RHL implementation to verify the entire front-end (spanning four intermediate languages), and our proofs involve no probabilistic reasoning (as interaction trees allow us to treat random sampling abstractly). A further benefit of our RHL is that, in spite of the more complex semantics, the correctness proofs are significantly shorter.

Contribution 3: Security Experiments and Preservation of Cryptographic Security. In Section 6, we formalize security experiments with interaction trees, and we provide instantiations of security experiments where the adversary interacts with either a Jasmin implementation or its compilation. We apply the compiler correctness theorem to prove that the adversary cannot distinguish between a security experiment based on a source Jasmin implementation and the same experiment based

on the compiled implementation. For concreteness, our theorem is stated for KEMs. Interestingly, our theorem underpins all assumptions that guarantee preservation of cryptographic security, including a new assumption about the well-initialization of return values. While retrospectively self-evident, this new assumption highlights the benefits of formally verified end-to-end security theorems in a unified framework.

Limitations. Our formalization does not yet cover the Jasmin compiler back-end, which comprises five passes, and is left for future work. Furthermore, preservation of cryptographic security is proven assuming the same adversary model for source and target programs—in contrast to robust compilation [1], which explicitly provisions for different adversary models between source and targets. However, one can connect with more concrete assembly-level adversarial models, as briefly discussed in the conclusion and will be reported separately.

Artifact. We provide a modified version of the Jasmin compiler with the new proofs for its front-end as an artifact in https://doi.org/10.5281/zenodo.17604303.

2 Interaction Trees

Interaction Trees [62] (ITrees) provide a convenient framework to reason about interactive and nonterminating computations. Informally, ITrees represent computations as potentially infinite trees, where each non-leaf node is either a silent step or a visible event. Interaction trees are an attractive option for us because, on the one hand, unlike most operational semantics, they offer an executable semantics (as a functional definition) and compositional (we can reason inductively on the language syntax rather than on the more complex semantic datatype). On the other hand, unlike old-style denotational semantics, ITrees support modularity (we can deal with different types of events separately), and they allow for a comparatively simple style of coinductive reasoning, supported by a rich equational theory and relying on parameterized coinduction [65].

2.1 Background on Interaction Trees

Formally, ITrees are modeled in Rocq as a coinductive type parameterized by a family of events $\mathsf{E} : \mathsf{Type} \to \mathsf{Type}$ and a type of results R:

$$\mathsf{itree}\ \mathsf{E}\ \mathsf{R} \ni t ::= \mathsf{Ret}(r) \mid \mathsf{Tau}(t) \mid \mathsf{Vis}(e, k),$$

where $r : \mathsf{R}$, $e : \mathsf{E}\ \mathsf{A}$, and $k : \mathsf{A} \to \mathsf{itree}\ \mathsf{E}\ \mathsf{R}$ for some type A. The *return* constructor $\mathsf{Ret}(r)$ labels the leaves of the tree: each leaf denotes a terminated computation that yields the specified result r. Internal nodes come in two flavors. First, *silent steps* $\mathsf{Tau}(t)$ perform an unobservable transition before continuing as t. Secondly, *effectful steps* $\mathsf{Vis}(e, k)$ trigger the visible event e and expect an answer $a : \mathsf{A}$, which is given to the continuation k to produce one branch per answer. Conceptually, visible events are deferred computations whose meaning will be supplied later by an event handler.

Interpretation. Interaction trees can be interpreted modularly with respect to the disjoint union of event families (written $\mathsf{E}_1 +' \mathsf{E}_2$) and the subevent relation (written $\mathsf{E}_1 \sqsubseteq \mathsf{E}_2$). The polymorphic operator $\mathsf{trigger} : \forall \mathsf{A}.\ \mathsf{E}'\ \mathsf{A} \to \mathsf{itree}\ \mathsf{E}\ \mathsf{A}$ allows lifting any subevent in E' to an ITree of E, given $\mathsf{E}' \sqsubseteq \mathsf{E}$. The $\mathsf{iter} : \forall \mathsf{A}\ I.\ (I \to M\ \ (I + \mathsf{A})) \to I \to M\ \mathsf{A}$ operator allows mapping an ITree to any monad M that supports iteration. ITrees are iterative monads, and iter can be seen as a while loop, repeatedly running its body (the first argument) until it produces a value of the result type A. Mappings from ITrees into a monad M can be defined via *event handlers*, which are functions $H : \forall \mathsf{A}.\ \mathsf{E}\ \mathsf{A} \to M\ \mathsf{A}$ for some event subfamily E. A generic interpreter operator, $\mathsf{interp} : (\forall \mathsf{A}.\ \mathsf{E}\ \mathsf{A} \to M\ \mathsf{A}) \to \forall \mathsf{A}.\ \mathsf{itree}\ \mathsf{E}\ \mathsf{A} \to M\ \mathsf{A}$, based on iter, can be used to fold a handler

$$\textsc{Ret}\ \frac{}{r_1\ \phi\ r_2}{\Big/}\ \mathsf{Ret}(r_1) \mathrel{\dot{\approx}} \mathsf{Ret}(r_2) \qquad \textsc{Tau}\ \frac{t_1 \mathrel{\dot{\approx}} t_2}{\mathsf{Tau}(t_1) \mathrel{\dot{\approx}} \mathsf{Tau}(t_2)} \qquad \boxed{\textsc{Cut}_L\ \frac{C_L(e_1)}{\mathsf{Vis}(e_1,k_1) \overset{u}{\mathrel{\dot{\approx}}} t_2}} \qquad \boxed{\textsc{Cut}_R\ \frac{C_R(e_2)}{t_1 \overset{u}{\mathrel{\dot{\approx}}} \mathsf{Vis}(e_2,k_2)}} \qquad \textsc{Tau}_L\ \frac{t_1 \mathrel{\dot{\approx}} t_2}{\mathsf{Tau}(t_1) \mathrel{\dot{\approx}} t_2}$$

$$\textsc{Tau}_R\ \frac{t_1 \mathrel{\dot{\approx}} t_2}{t_1 \mathrel{\dot{\approx}} \mathsf{Tau}(t_2)} \qquad \textsc{Vis}\ \frac{e_1\ I_P\ e_2 \qquad \forall a_1\ a_2.\ (e_1, a_1)\ I_Q\ (e_2, a_2) \implies k_1(a_1) \mathrel{\dot{\approx}} k_2(a_2)}{\mathsf{Vis}(e_1,k_1) \mathrel{\dot{\approx}} \mathsf{Vis}(e_2,k_2)}$$

Fig. 1. Bisimulation relations $\overset{e}{\approx}$ and $\overset{u}{\approx}$. The boxed rules are for $\overset{u}{\approx}$ only, all other are valid for both $\overset{e}{\approx}$ and $\overset{u}{\approx}$.

corecursively on an ITree. When the event handler itself is recursive, we can use the operator interp_mrec : (∀A. E′ A → itree (E′ +′ E) A) → ∀A. itree (E′ +′ E) A → itree E A. Since ITrees can be themselves interpretation targets, we can write modular interpreters, each for a subfamily of events, and compose them sequentially.

Bisimulation. A basic generalization of weak bisimilarity for ITrees, denoted ≈, was introduced in [62] as a coinductive-inductive relation, parametrized by a relation over result. However, ≈ can only relate ITrees that have exactly the same events. In our development we rely on a generalization, known as *heterogeneous equivalence* and denoted $\overset{e}{\approx}$, originally introduced in [56] and extensively used in [23]. Heterogeneous equivalence allows relating ITrees that depend on distinct return types (denoted R_1 and R_2) and distinct event families (denoted E_1 and E_2). It is defined in Rocq as a coinductive-inductive relation in the style of [62], as shown in Figure 1. We follow [44] in distinguishing between coinductive uses of the premise and inductive (or simply non-coinductive) ones by means of double and single bars, respectively. It depends on a relation over results $\phi : \mathsf{R}_1 \to \mathsf{R}_2 \to \mathsf{Prop}$, as well as on a *handler contract* (I_P, I_Q), where $I_P : \forall \mathsf{A}_1\mathsf{A}_2.\ \mathsf{E}_1\ \mathsf{A}_1 \to \mathsf{E}_2\ \mathsf{A}_2 \to \mathsf{Prop}$ is the *event precondition* and the *event postcondition* is $I_Q : \forall \mathsf{A}_1\mathsf{A}_2.\ \mathsf{E}_1\ \mathsf{A}_1 \to \mathsf{A}_1 \to \mathsf{E}_2\ \mathsf{A}_2 \to \mathsf{A}_2 \to \mathsf{Prop}$. The Ret rule relates any pair of ϕ-related return values. The Vis rule requires that, given answer types A_1 and A_2 and events $e_1 : \mathsf{E}_1\ \mathsf{A}_1$ and $e_2 : \mathsf{E}_2\ \mathsf{A}_2$, the events are related by I_P. The continuations $k_1(a_1)$ and $k_2(a_2)$ must also be in relation, assuming that the answers a_1 and a_2 satisfy the event postcondition I_Q. The $\textsc{Tau}_L$ and $\textsc{Tau}_R$ rules rely on their hypothesis *inductively* rather than coinductively, ensuring that related ITrees can differ only up to a *finite* number of silent transition steps. When both I_P and I_Q are equalities, $\overset{e}{\approx}_{I_P,I_Q,\phi}$ is provably equivalent to $\approx_\phi$.

2.2 Equivalence up-to-Cutoff

We introduce a generalization of heterogeneous equivalence to support error handling in terms of undefined behavior. This generalization, denoted $\overset{u}{\approx}$, relies on the notion of *cutoff* as designated event, on either side of the relation, that allows the relation to hold trivially from that point, regardless of the continuation. Its coinductive-inductive definition, also shown in Figure 1, extends that of $\overset{e}{\approx}$ with two additional boolean predicate parameters, $C_L : \forall \mathsf{A}.\ \mathsf{E}_1\ \mathsf{A} \to \mathtt{bool}$ and C_R of analogous type for E_2, which decide which events are cutoffs on each side of the relation. The $\textsc{Cut}_L$ rule relates $\mathsf{Vis}(e,k)$: itree E_1 R_1 with any t : itree E_2 R_2 whenever e is a cutoff event on the left-hand side; the $\textsc{Cut}_R$ rule is defined symmetrically. We prove that when there are no cutoffs on either side, $\overset{u}{\approx}$ is equivalent to $\overset{e}{\approx}$.

$$\begin{aligned}
\mathsf{Expr} \ni e &::= n \mid b \mid x \mid (e, \cdots, e) \mid \oplus(e) \mid x[e] \mid [e] \\
\mathsf{LVal} \ni \ell &::= x \mid x[e] \mid [e] \mid (\ell, \cdots, \ell) \\
\mathsf{Comm} \ni c &::= \mathbf{skip} \mid \ell \coloneqq e \mid \ell \coloneqq \mathbf{rand}(e) \mid c\,;c \mid \mathbf{if}\ (e)\ \{c\}\ \{c\} \mid \mathbf{while}\ (e)\ \{c\} \mid \ell \coloneqq f(e)
\end{aligned}$$

Fig. 2. Core syntax of Jasmin.

$$\begin{aligned}
(\!|\mathbf{skip}|\!)^{\mathsf{cer}}_{\sigma} &\triangleq \mathsf{Ret}(\sigma) \\
(\!|\ell \coloneqq e|\!)^{\mathsf{cer}}_{\sigma} &\triangleq v \leftarrow (\!|e|\!)^{\mathsf{Expr}}_{\sigma}\,; (\!|\ell \leftarrow v|\!)^{\mathsf{LVal}}_{\sigma} \\
(\!|\ell \coloneqq \mathbf{rand}(e)|\!)^{\mathsf{cer}}_{\sigma} &\triangleq n \leftarrow (\!|e|\!)^{\mathsf{Expr}}_{\sigma}\,; a \leftarrow \mathsf{trigger}(\mathsf{Rnd}(n))\,; (\!|\ell \leftarrow a|\!)^{\mathsf{LVal}}_{\sigma} \\
(\!|c\,;c'|\!)^{\mathsf{cer}}_{\sigma} &\triangleq \sigma' \leftarrow (\!|c|\!)^{\mathsf{cer}}_{\sigma}\,; (\!|c'|\!)^{\mathsf{cer}}_{\sigma'} \\
(\!|\mathbf{if}\ (e)\ \{c_\top\}\ \{c_\bot\}|\!)^{\mathsf{cer}}_{\sigma} &\triangleq b \leftarrow (\!|e|\!)^{\mathsf{Expr}}_{\sigma}\,; (\!|c_b|\!)^{\mathsf{cer}}_{\sigma} \\
(\!|\mathbf{while}\ (e)\ \{c\}|\!)^{\mathsf{cer}}_{\sigma} &\triangleq \mathsf{iter}\left(\lambda\sigma.\, b \leftarrow (\!|e|\!)^{\mathsf{Expr}}_{\sigma}\,; \begin{cases} \sigma' \leftarrow (\!|c|\!)^{\mathsf{cer}}_{\sigma}\,; \mathsf{Ret}(\mathsf{inl}(\sigma')) & \text{if } b, \\ \mathsf{Ret}(\mathsf{inr}(\sigma)) & \text{otherwise} \end{cases}\right)(\sigma) \\
(\!|\ell \coloneqq f(e)|\!)^{\mathsf{cer}}_{\sigma} &\triangleq v \leftarrow (\!|e|\!)^{\mathsf{Expr}}_{\sigma}\,; (m', v') \leftarrow \mathsf{trigger}(\mathsf{Call}(f, \sigma_m, v))\,; (\!|\ell \leftarrow v'|\!)^{\mathsf{LVal}}_{(m', \sigma_r)}
\end{aligned}$$

Fig. 3. Interaction tree semantics of Jasmin commands.

3 Interaction Tree Semantics for Jasmin

This section introduces the Jasmin language and presents its new semantics based on interaction trees. The semantics is defined in two steps, as is standard in the ITree literature.

Figure 2 presents the core syntax of Jasmin from this perspective. Here, e is an expression, n is a natural number, b is a boolean, $x \in \mathsf{Var}$ is a variable, $(e, \cdots, e)$ is a tuple of expressions, $\oplus$ is an operator (e.g., $+$, $\leq$, or $\wedge$), ℓ is a left-value, $x[e]$ is an array access to x at position e, $[e]$ is a memory access dereferencing the address e, and f is a function name.

Mutable states, denoted State, are pairs of a variable map $r : \mathsf{Var} \to \mathsf{Val}$ that assigns values to variables, and a memory $m : \{0,1\}^{64} \to \{0,1\}^{8}$ that maps addresses to bytes. Given a state σ, the projections σ_r and σ_m are its variable map and memory, respectively.

The *intraprocedural* semantics of Jasmin $(\!|c|\!)^{\mathsf{cer}}_{\sigma} : \mathsf{itree}\ (\mathsf{Call} +' \mathsf{Err} +' \mathsf{Rnd})\ \mathsf{State}$, where c is a command and σ a state, employs three event families: Call for function calls, Rnd for random sampling, and Err for errors. Additionally, it relies on the expected semantics of expressions $(\!|e|\!)^{\mathsf{Expr}}_{\sigma} : \mathsf{itree}\ \mathsf{Err}\ \mathsf{Val}$ and left-values $(\!|\ell \leftarrow v|\!)^{\mathsf{LVal}}_{\sigma} : \mathsf{itree}\ \mathsf{Err}\ \mathsf{State}$ where v is a value and σ is a state, where Err is an error event.

Figure 3 presents the intraprocedural semantics. The **skip** command returns the state unchanged. An assignment evaluates the right-hand side and writes the result to the state—note that either step can trigger an Err event. A random sampling $\ell \coloneqq \mathbf{rand}(e)$ evaluates e to a number n, triggers a $\mathsf{Rnd}(n)$ event that expects n uniformly sampled bytes, and writes them to ℓ. The cases of sequencing and conditionals use the standard ITree bind operator. While loops use the iter operator to iteratively execute the body, potentially infinitely many times. Finally, function calls evaluate their argument e, trigger a Call event, write the answer to ℓ, and update the memory. Each Call event carries function name, memory, and argument value.

The *interprocedural semantics* is obtained by interpreting Call events, which we handle by executing the body of the callee. We define the handler for Call events as follows:

$$H^{\mathsf{c}}(\mathsf{Call}(f, m, v)) \triangleq \sigma \leftarrow (\!|f_{\mathsf{body}}|\!)^{\mathsf{cer}}_{\sigma_m, \{f_{\mathsf{arg}} \mapsto v\}} ; \mathsf{Ret}(\sigma_m, \sigma_r(f_{\mathsf{res}})),$$

where $\{f_{\mathsf{arg}} \mapsto v\}$ is the variable map that contains v as the argument of f, and f_{body} and f_{res} are the body and the results of f, respectively. With this handler and the mutual recursion operator interp_mrec, we can define the interprocedural semantics of a command c and a state σ as follows:

$$(\!|c|\!)^{\mathsf{er}}_{\sigma} : \mathsf{itree}\ (\mathsf{Err} +' \mathsf{Rnd})\ \mathsf{State} \triangleq \mathsf{interp_mrec}\big(H^{\mathsf{c}}, (\!|c|\!)^{\mathsf{cer}}_{\sigma}\big),$$

which consumes all Call events. The Err and Rnd events, on the other hand, remain uninterpreted, as effects of the program.

4 Relational Hoare Logic for an Interaction Trees Semantics

In this section, we present a Relational Hoare Logic for ITree-based program semantics. We first present a vanilla logic and then discuss a specialized version for compiler correctness proofs, which we later use to verify the Jasmin compiler.

4.1 Tuples and Validity

Relational Hoare Logic tuples relate two programs with respect to pre- and postconditions. The definitions of RHL tuple uses the ITree equivalence $\overset{e}{\approx}$, presented in Section 2.

Definition 1 (Valid RHL Command Tuple). We write $I_P, I_Q \models c_1 \sim c_2 : \Phi \Rightarrow \Psi$ when for every pair of states σ_1 and σ_2,

$$\sigma_1\ \Phi\ \sigma_2 \implies (\!|c_1|\!)^{\mathsf{cer}}_{\sigma_1} \overset{e}{\approx}_{I_P, I_Q, \Psi} (\!|c_2|\!)^{\mathsf{cer}}_{\sigma_2},$$

where c_1 and c_2 are commands and $\Phi, \Psi : \mathsf{State} \to \mathsf{State} \to \mathsf{Prop}$ are the pre- and postconditions on states, respectively. Note that the handler contract I_P and I_Q relate Call, Err, and Rnd events.

Intuitively, a command tuple is valid when running the commands c_1 and c_2 on a pair of states that satisfy the precondition Φ gives behaviors that are related by $\overset{e}{\approx}$ with respect to the postcondition Ψ and the contract I_P and I_Q.

Definition 2 (Valid RHL Program Tuple). We write $I_P, I_Q \models P_1 \sim P_2 : \Phi \Rightarrow \Psi$ when for every pair of functions f and g and states σ_1 and σ_2,

$$\sigma_1\ \widehat{\Phi}_{f,g}\ \sigma_2 \implies (\!|f_{\mathsf{body}}|\!)^{\mathsf{er}}_{\sigma_1} \overset{e}{\approx}_{I_P, I_Q, \widehat{\Psi}_{f,g}} (\!|g_{\mathsf{body}}|\!)^{\mathsf{er}}_{\sigma_2},$$

where f_{body} and g_{body} are the bodies of functions f and g respectively in P_1 and P_2, Φ and Ψ are a *relational specification* for functions (i.e., families of relations on memories and values, indexed by pairs of function names), and the handler contract I_P and I_Q is defined for Err and Rnd events. We write $\widehat{\Phi}_{f,g}$ (resp. $\widehat{\Psi}_{f,g}$) for the relation that holds on initial (resp. final) states whose arguments (resp. results) are related by $\Phi_{f,g}$ (resp. $\Psi_{f,g}$).

In the following, we use $\models e_1 \sim e_2 : \Phi \Rightarrow \psi$ (where e_1 and e_2 are expressions, Φ relates states, and ψ relates values) to express that evaluating e_1 and e_2 in Φ-related states yields ψ-related values. We use $\models \ell_1 \sim \ell_2 : \Phi \mid \phi \Rightarrow \Psi$ (where Φ and Ψ relate states and ϕ values) when writing ϕ-related values to Φ-related states yields Ψ-related states.

$$\textsc{Skip}\ \frac{}{\vdash \mathbf{skip} \sim \mathbf{skip} : \Phi \Rightarrow \Phi}$$

$$\textsc{Seq}\ \frac{\vdash c_1 \sim c_2 : \Phi \Rightarrow \Phi' \qquad \vdash c_1' \sim c_2' : \Phi' \Rightarrow \Phi''}{\vdash c_1 ; c_1' \sim c_2 ; c_2' : \Phi \Rightarrow \Phi''}$$

$$\textsc{Assign}\ \frac{\models e_1 \sim e_2 : \Phi \Rightarrow \phi \qquad \models \ell_1 \sim \ell_2 : \Phi \mid \phi \Rightarrow \Psi}{\vdash \ell_1 \coloneqq e_1 \sim \ell_2 \coloneqq e_2 : \Phi \Rightarrow \Psi}$$

$$\textsc{Case}\ \frac{\models e_1 \sim e_2 : \Phi \Rightarrow = \qquad \vdash c_1 \sim c_2 : \Phi \wedge e_1 \Rightarrow \Psi \qquad \vdash c_1' \sim c_2' : \Phi \wedge \neg e_1 \Rightarrow \Psi}{\vdash \mathbf{if}\ (e_1)\ \{c_1\}\ \{c_1'\} \sim \mathbf{if}\ (e_2)\ \{c_2\}\ \{c_2'\} : \Phi \Rightarrow \Psi}$$

$$\textsc{While}\ \frac{\models e_1 \sim e_2 : \Phi \Rightarrow = \qquad \vdash c_1 \sim c_2 : \Phi \wedge e_1 \Rightarrow \Phi}{\vdash \mathbf{while}\ (e_1)\ \{c_1\} \sim \mathbf{while}\ (e_2)\ \{c_2\} : \Phi \Rightarrow \Phi \wedge \neg e_1}$$

$$\textsc{Rand}\ \frac{\models e_1 \sim e_2 : \Phi \Rightarrow I_P \qquad \models \ell_1 \sim \ell_2 : \Phi \mid I_Q \Rightarrow \Psi}{\vdash \ell_1 \coloneqq \mathbf{rand}(e_1) \sim \ell_2 \coloneqq \mathbf{rand}(e_2) : \Phi \Rightarrow \Psi}$$

$$\textsc{Call}\ \frac{\models e_1 \sim e_2 : \left(\Phi_r \wedge I_P^m\right) \Rightarrow I_P^v \qquad \models \ell_1 \sim \ell_2 : \left(\Phi_r \wedge I_Q^m\right) \mid I_Q^v \Rightarrow \Psi}{\vdash \ell_1 \coloneqq f(e_1) \sim \ell_2 \coloneqq g(e_2) : \left(\Phi_r \wedge I_P^m\right) \Rightarrow \Psi}$$

$$\textsc{Assign}_L\ \frac{\forall \sigma_1 \sigma_1' \sigma_2.\ \sigma_1\ \Phi\ \sigma_2 \wedge (\!|\ell \coloneqq e|\!)_{\sigma_1}^{\mathsf{cer}} \approx \mathsf{Ret}(\sigma_1') \implies \sigma_1'\ \Psi\ \sigma_2}{\vdash \ell \coloneqq e \sim \mathbf{skip} : \Phi \Rightarrow \Psi}$$

$$\textsc{Case}_L\ \frac{\vdash c_1 \sim c_2 : \left(\lambda \sigma_1 \sigma_2.\ \sigma_1\ \Phi\ \sigma_2 \wedge (\!|e|\!)_{\sigma_1}^{\mathsf{Expr}} \approx \mathsf{Ret}(\top)\right) \Rightarrow \Psi \qquad \vdash c_1' \sim c_2 : \left(\lambda \sigma_1 \sigma_2.\ \sigma_1\ \Phi\ \sigma_2 \wedge (\!|e|\!)_{\sigma_1}^{\mathsf{Expr}} \approx \mathsf{Ret}(\bot)\right) \Rightarrow \Psi}{\vdash \mathbf{if}\ (e)\ \{c_1\}\ \{c_1'\} \sim c_2 : \Phi \Rightarrow \Psi}$$

$$\textsc{Conseq}\ \frac{I_P', I_Q' \vdash c_1 \sim c_2 : \Phi' \Rightarrow \Psi' \qquad \Phi \implies \Phi' \qquad I_P' \implies I_P \qquad \Psi' \implies \Psi \qquad I_Q \implies I_Q'}{I_P, I_Q \vdash c_1 \sim c_2 : \Phi \Rightarrow \Psi}$$

$$\textsc{Trans}\ \frac{I_{P1}, I_{Q_1} \vdash c_1 \sim c_2 : \Phi_1 \Rightarrow \Psi_1 \qquad I_{P2}, I_{Q_2} \vdash c_2 \sim c_3 : \Phi_2 \Rightarrow \Psi_2}{I_{P1} \circ I_{P2}, I_{Q_1} \circ I_{Q_2} \vdash c_1 \sim c_3 : \Phi_1 \circ \Phi_2 \Rightarrow \Psi_1 \circ \Psi_2}$$

$$\textsc{Rec}\ \frac{I_P \cup \Phi, I_Q \cup \Psi \vdash f_{\mathrm{body}} \sim g_{\mathrm{body}} : \widehat{\Phi}_{f,g} \Rightarrow \widehat{\Psi}_{f,g} \quad \text{for every } f, g}{I_P, I_Q \vdash P_1 \sim P_2 : \Phi \Rightarrow \Psi}$$

Fig. 4. Relational Hoare Logic.

4.2 Proof Rules

Relational Hoare Logic rules typically come in three flavors: *two-sided* rules, *one-sided* rules, and *structural* rules. This section discusses each group in turn. As usual, we write $I_P, I_Q \vdash c_1 \sim c_2 : \Phi \Rightarrow \Psi$ (resp. $I_P, I_Q \vdash P_1 \sim P_2 : \Phi \Rightarrow \Psi$) when an RHL command (resp. program) tuple is derivable. We leave I_P and I_Q implicit when they are clear from the context.

Two-Sided Rules. The first seven rules of Figure 4 are two-sided rules. The Skip and Seq rules are analogous to their Hoare Logic counterparts. The Assign rule states that two assignments are related if evaluating their expressions e_1 and e_2 under the assumption Φ yields ϕ-related values, and writing such values in ℓ_1 and ℓ_2 ensures Ψ.

The Case rule is for conditionals. It is a *synchronous* rule, i.e., it requires that the guards e_1 and e_2 evaluate to the same value, hence we use equality as the postcondition for the expressions. Analogously to its Hoare Logic counterpart, the other two premises are that (pairs of) both branches guarantee the postcondition, assuming the precondition, where $\Phi \wedge e_1$ denotes that both Φ and e_1 hold. The While rule is analogous: we require that the guards are equal both initially and after the loop body. The formula Φ acts as a loop invariant, like in Hoare Logic.

The Rand rule deals with random sampling. It requires that the arguments e_1 and e_2 satisfy the handler precondition I_P (for Rnd events), and that the handler postcondition I_Q ensures the postcondition Ψ on the states after writing the answers to ℓ_1 and ℓ_2.

Finally, the Call rule relates two function calls. The specification for functions is given by the handler contract I_P and I_Q for Call events: it establishes the (relational) pre- and postconditions for calling each pair of functions. As such, I_P and I_Q are relations between pairs of lists of values and memories. Without loss of generality, we split them into I_P^v and I_P^m, and I_Q^v and I_Q^m, for the relations values and memories, respectively.

The first premise of the Call rule states that, given a precondition on variable maps Φ_r, the arguments e_1 and e_2 must satisfy the functions' precondition I_P^v. The second premise asserts the postcondition Ψ after writing the results in states where *the initial* precondition on variable maps Φ_r holds, as well as the functions' postcondition I_Q. The precondition Φ_r holds after writing results because the callee executes on its own local variable map, leaving the caller's unchanged.

One-Sided Rules. These rules relate structurally dissimilar programs—in most cases, an event-free command with **skip**. The $\textsc{Assign}_L$ and $\textsc{Case}_L$ rules in Figure 4 are one-sided rules.

The $\textsc{Assign}_L$ rule relates an assignment on the left-hand side with **skip** on the right-hand side. The premise states that the postcondition Ψ must hold after an assignment on the state on the left-hand side. The $\textsc{Case}_L$ rule relates a conditional on the left-hand side with an arbitrary command c_2 on the right-hand side. Each premise relates a branch of the conditional with c_2.

Structural Rules. Our logic satisfies many standard structural rules of program logics, such as the Conseq and Trans rules in Figure 4. The rule of consequence (or weakening), Conseq, allows us to strengthen the precondition (resp. the event postcondition) and weaken the postcondition (resp. the event precondition) of an RHL judgment. The Trans rule allows us to compose judgments transitively. We use $\circ$ for the composition of two relations, defined as $x\ (R \circ R')\ z \triangleq \exists y.\ x\ R\ y \wedge y\ R'\ z$.

Recursion and Programs. Figure 4 presents the Rec rule to relate two programs. The rule lifts and generalizes the classical recursion rule in Hoare Logic [9] to the relational setting. As before, we write f_{body} and g_{body} for the bodies of f and g in P_1 and P_2, respectively. In the premise, we extend the contract to $I_P \cup \Phi$ and $I_Q \cup \Psi$, which relates Call events using Φ and Ψ, and other events using I_P and I_Q. The rule requires proving that *the bodies* of every pair of functions f and g satisfy their relational specification *assuming that all function calls satisfy theirs*. Note that this allows reasoning about mutually recursive functions.

As a result, two programs are related if all pairs of bodies satisfy their relational specification. While the premise of this rule involves only the *intraprocedural semantics* $(\!|\cdot|\!)^{\text{cer}}$, the soundness theorem below shows that its conclusion relates the *interprocedural semantics* $(\!|\cdot|\!)^{\text{er}}$ of functions.

Soundness. The following theorem establishes the soundness of our program logic, relating the intra- and interprocedural semantics.

Theorem 1 (Soundness). *If P_1 and P_2 are related, i.e., $I_P, I_Q \vdash P_1 \sim P_2 : \Phi \Rightarrow \Psi$, then every pair of functions satisfy their specification, i.e., $I_P, I_Q \models P_1 \sim P_2 : \Phi \Rightarrow \Psi$.*

Proof. This theorem follows naturally from the properties of interp_mrec. It is formalized as `wequiv_fun_ind` in `proofs/lang/relational_logic.v`. □

4.3 Specializing the Logic for Compiler Proofs

Verifying a realistic compiler with RHL can benefit significantly from using a specialized notion of validity and specialized rules that capture common compiler patterns. This section presents

extensions to RHL that proved particularly helpful in proving the Jasmin compiler. We distill four key ingredients of our specialized logic, which are not specific to Jasmin or to its ITree semantics.

Equivalence up-to-Cutoff. Compiler correctness typically relates the semantics of a source program and that of its compilation *as long as the source program is safe.* Assuming safety is often essential in compiler proofs, as it allows us to disregard invalid behaviors that need not be preserved. Additionally, safety also encodes important invariants of the compiler passes (e.g., if the compiler introduces a memory access, that access must be in-bounds) that subsequent passes rely upon.

We internalize this kind of "up to source safety" reasoning in our logic by redefining the validity of RHL tuples with equivalence up-to-cutoff $\overset{u}{\approx}$, instantiating the left cutoff to be Err events and no cutoffs on the right-hand side. Changing the relation in RHL tuples impacts the interpretation of the judgments for commands, expressions, and left-values. For instance, the Assign rule now states that if either e_1 or ℓ_1 trigger an error, we can choose an arbitrary postcondition—including a contradiction—and relate the assignments vacuously. All rules admit this line of reasoning. The soundness of the proof system extends directly to this new setting.

Parameterization with Respect to Semantics. Compiler passes frequently have different input and output languages (e.g., transforming a program with abstract arrays into one with a concrete memory layout)—indeed, the Jasmin front-end contains four such passes. To verify these passes, our Rocq formalization of the semantics and of RHL is substantially more general that what was presented in the previous section, *being parametric on the semantics of either side.* This generalization allows us to verify the front-end of Jasmin with *a single implementation* of RHL, and is essential in the proof of the Inlining pass, as we discuss in Section 5.2. We omit this more general form for brevity and refer the reader to the Rocq development for details.

Specialized One-Sided Rules. Compiler passes often introduce or remove instructions from the program. In such cases, the source and target programs are not in lock-step, and cannot be related using two-sided rules. One-sided rules allow us to relate such programs: for example, in a compiler pass that removes dead code, we can relate an assignment to a dead variable with the **skip** command using the rule $\textsc{Assign}_L$, since the postcondition of that pass requires that the states coincide on live variables, and said assignment satisfies it trivially.

Unfortunately, the one-sided rules in Figure 4 are too generic and become cumbersome to use directly for more complex transformations. Thus, we introduce new one-sided rules tailored for common compiler transformations. These rules pattern-match on the command on the left-hand side (i.e., the source) and set the command on the right-hand side (i.e., the target) to the transformation of the former. The rules have non-relational proof obligations to prove that the right-hand side correctly implements the source. Such obligations enjoy good automation in the Rocq prover, thanks to our defining a partial evaluator for commands that can reduce loop- and event-free commands to their results; for instance, it establishes that $(\!|x \coloneqq 0|\!)^{\mathrm{cer}}_{\sigma} \approx \mathrm{Ret}(\sigma[x \mapsto 0])$.

The first four rules in Figure 5 are specialized one-sided rules. The $\textsc{Assign}_L$ rule relates an assignment to an arbitrary command. This rule is well-suited for passes that expand an assignment into several ones (e.g., mapping a complex load $x \coloneqq a[y + z]$ into two instructions $t \coloneqq y + z\,; x \coloneqq a[t]$) since our partial evaluator can readily reduce the assignments to their result.

The $\textsc{Case}_L$ rule allows us to consume a "prologue" command on the right-hand side before relating two conditionals. The first premise ensures that the guards e_1 and e_2 are synchronized and that the prelude c is event-free, safe, and terminating. This pattern applies to passes that introduce a prologue before a conditional to compute a complex guard; for instance, the Instruction Selection pass in Jasmin maps conditionals to a comparison and the appropriate combination of flags. For instance, it transforms $\mathbf{if}\ (x \leq y)\ \{\ldots\}\ \{\ldots\}$ into $o, s, c, z \coloneqq \texttt{\#CMP}(x, y)\,;\mathbf{if}\ (c \vee z)\ \{\ldots\}\ \{\ldots\}$, since

$$\textsc{Assign}_L\ \frac{\forall \sigma_1 \sigma_1' \sigma_2.\ \sigma_1\ \Phi\ \sigma_2 \wedge (\!|\ell := e|\!)^{\mathsf{cer}}_{\sigma_1} \approx \mathsf{Ret}(\sigma_1') \implies \exists \sigma_2'.\ (\!|c|\!)^{\mathsf{cer}}_{\sigma_2} \approx \mathsf{Ret}(\sigma_2') \wedge \sigma_1'\ \Psi\ \sigma_2'}{\vdash \ell := e \sim c : \Phi \Rightarrow \Psi}$$

$$\textsc{Case}_L\ \frac{\begin{array}{c}\forall v \sigma_1 \sigma_2.\ \sigma_1\ \Phi\ \sigma_2 \wedge (\!|e_1|\!)^{\mathsf{Expr}}_{\sigma_1} \approx \mathsf{Ret}(v) \implies \exists \sigma_2'.\ \sigma_1\ \Phi'\ \sigma_2' \wedge (\!|c|\!)^{\mathsf{cer}}_{\sigma_2} \approx \mathsf{Ret}(\sigma_2') \wedge (\!|e_2|\!)^{\mathsf{Expr}}_{\sigma_2'} \approx \mathsf{Ret}(v) \\ \vdash c_1 \sim c_2 : \Phi' \Rightarrow \Psi \qquad \vdash c_1' \sim c_2' : \Phi' \Rightarrow \Psi\end{array}}{\vdash \textbf{if}\ (e_1)\ \{c_1\}\ \{c_1'\} \sim c\,;\textbf{if}\ (e_2)\ \{c_2\}\ \{c_2'\} : \Phi \Rightarrow \Psi}$$

$$\textsc{Case}_b\ \frac{\forall \sigma_1 \sigma_2.\ \sigma_1\ \Phi\ \sigma_2 \implies (\!|e|\!)^{\mathsf{Expr}}_{\sigma_1} \approx \mathsf{Ret}(b)}{\vdash \textbf{if}\ (e)\ \{c_\top\}\ \{c_\bot\} \sim c_b : \Phi \Rightarrow =}$$

$$\textsc{Unroll}_L\ \frac{\vdash \textbf{if}\ (e)\ \{c\,;\textbf{while}\ (e)\ \{c\}\}\ \{\textbf{skip}\} \sim c' : \Phi \Rightarrow \Psi}{\vdash \textbf{while}\ (e)\ \{c\} \sim c' : \Phi \Rightarrow \Psi}$$

$$\textsc{Assign}_{\mathsf{chk}}\ \frac{\mathsf{chk}_e(\alpha, e_1, e_2, \alpha') \qquad \mathsf{chk}_\ell(\alpha', \ell_1, \ell_2, \alpha'')}{\vdash \ell_1 := e_1 \sim \ell_2 := e_2 :\ R_\alpha\ \Rightarrow\ R_{\alpha''}}$$

$$\textsc{Case}_{\mathsf{chk}}\ \frac{\begin{array}{c}\mathsf{chk}_e(\alpha, e_1, e_2, \alpha') \qquad \models e_1 \sim e_2 :\ R_\alpha\ \Rightarrow = \\ \vdash c_1 \sim c_2 :\ R_{\alpha'}\ \Rightarrow\ R_{\alpha''} \qquad \vdash c_1' \sim c_2' :\ R_{\alpha'}\ \Rightarrow\ R_{\alpha''}\end{array}}{\vdash \textbf{if}\ (e_1)\ \{c_1\}\ \{c_1'\} \sim \textbf{if}\ (e_2)\ \{c_2\}\ \{c_2'\} :\ R_\alpha\ \Rightarrow\ R_{\alpha''}}$$

Fig. 5. Specialized Relational Hoare Logic.

the disjunction of the carry and zero flags represent $\leq$ in x86-64. We have also formalized an analogous rule for loops.

Lastly, the $\textsc{Case}_b$ and $\textsc{Unroll}_L$ rules are useful in simplifying conditionals and loops with constant guards. The former relates such a conditional to the appropriate branch, and the latter relates a while loop to its one-step unrolling. These rules are used in Jasmin, for example, as part of constant branch elimination.

Leveraging Static Analyses in RHL. Many compiler passes leverage the results of program analyses to modify programs in a structure-preserving way. At an abstract level, such passes replace expressions (resp. left-values) with new expressions (resp. left-values) that are equivalent given the results of the analysis. Consequently, we have developed a general approach to validate such passes using simplified RHL rules.

Our approach entails an abstract set $\mathcal{A}$, a logical interpretation $(R_\alpha)_{\alpha \in \mathcal{A}} : \mathsf{State} \times \mathsf{State} \to \mathsf{Prop}$, and two *checkers* $\mathsf{chk}_e : \mathcal{A} \times \mathsf{Expr} \times \mathsf{Expr} \times \mathcal{A} \to \mathsf{Prop}$ and $\mathsf{chk}_\ell : \mathcal{A} \times \mathsf{LVal} \times \mathsf{LVal} \times \mathcal{A} \to \mathsf{Prop}$. Informally, the abstract set collects information about the two program executions, and the interpretation maps elements of the abstract domain to a logical representation. The checkers, on the other hand, verify whether an abstract value validates a desired relation $\mathcal{S}$ (for example, that two given expressions evaluate to the same value) and establish an updated abstract value. We identify three properties needed to incorporate checkers in the logic, namely:

$$\sigma_1\ R_\alpha\ \sigma_2 \wedge \mathsf{chk}_e(\alpha, e_1, e_2, \alpha') \implies \sigma_1\ R_{\alpha'}\ \sigma_2, \tag{Mono}$$

$$\mathsf{chk}_e(\alpha, e_1, e_2, \alpha') \implies \models e_1 \sim e_2 : R_\alpha \Rightarrow \mathcal{S}, \tag{Correct$_e$}$$

$$\mathsf{chk}_\ell(\alpha, \ell_1, \ell_2, \alpha') \implies \models \ell_1 \sim \ell_2 : R_\alpha \mid \mathcal{S} \Rightarrow R_{\alpha'}. \tag{Correct$_\ell$}$$

The first property ensures that no information is lost after analyzing expressions (since evaluating expressions does not change the state) and allows composing the results of multiple checks. The correctness of chk_e ensures that evaluating e_1 and e_2 in R_α-related states yields $\mathcal{S}$-related values.

The correctness of chk_ℓ ensures that if the check with α is successful and yields α', updating α-related states with $\mathcal{S}$-related values yields α'-related states.

The properties of checkers can be used to simplify several of the RHL rules presented in Figure 4. For instance, the last two rules of Figure 5 are simplified versions of the Assign and While rules, whose premises rely on the checkers to establish the necessary RHL tuples. Such premises are trivially discharged in compiler transformations that replace commands only when the checkers succeed, which is the case for many passes in Jasmin.

5 Verifying the Jasmin Compiler Front-End

In this section, we describe the new proof of correctness for the Jasmin compiler front-end. Crucially, *we did not modify the functionality of the compiler*, i.e., the compiler before and after our changes generates the same assembly. We begin with a primer on Jasmin and its compiler, followed by a high-level overview of the compiler correctness statement and an in-depth discussion of proofs of selected passes, including a discussion on potential extensions of our logic. Lastly, we compare with the previous proof effort and present statistics on the reduction in proof effort as a result of this work.

5.1 The Jasmin Framework and Compiler

Jasmin [5, 7] is a framework for high-assurance high-speed cryptography. The framework provides a programming language and compiler, on which we focus in this work. The language features a combination of low-level constructs (such as vectorized instructions) and high-level constructs (such as structured control flow, e.g., conditionals, loops, and functions), and zero-cost abstractions (such as explicit variable names and functional arrays). The latter are called zero-cost because they introduce no run-time overhead; for example, functional array manipulations are always compiled to in-place memory operations that involve no copies.

The compiler supports the x86-64, ARMv7, and RISC-V architectures. It is designed to be predictable and provide a large degree of control to the programmer; thus, its primary purpose is to eliminate the zero-cost abstractions. The compiler also minimizes stack usage through array reuse, allowing two arrays with different live ranges to share the same stack region. Figure 6 presents the passes of the Jasmin compiler. Ellipses correspond to the different representations of programs, rectangles to transformations, and dotted rectangles to checkers—i.e., passes that leave the program unchanged but may reject it. Red boxes correspond to trusted steps, green ones to those verified in Rocq directly, and blue ones to steps verified using translation validation. The passes inside the dashed green box (i.e., second to sixth rows) are the compiler *front-end*, which is the focus of our work, whereas the remaining passes are the back-end. Section A presents a summary of the functionality of each pass.

Figure 6 shows that the compiler front-end has four internal languages: Jasmin (the source), Subword, Direct, and Stack. These languages have the same syntax but different semantics, i.e., a language construct such as a function call has different effects, becoming lower-level down the compilation chain. Similarly, their semantics are instances of a general one, parameterized over the effects of certain constructs, such as function calls. Using instances of a general syntax and semantics as internal languages enables extensive sharing of code and proof infrastructure.

Let us now overview the functionality of the three compiler passes we discuss below:

(1) The Register Renaming pass renames variables to use machine register names; for example, $x := y + 3$ gets renamed to $\mathsf{rdi} := \mathsf{rdi} + 3$. This pass performs no spilling, and therefore leaves the program structure unchanged. We verify this pass in translation validation style: we use an unverified algorithm implemented in OCaml to generate a renamed program, and

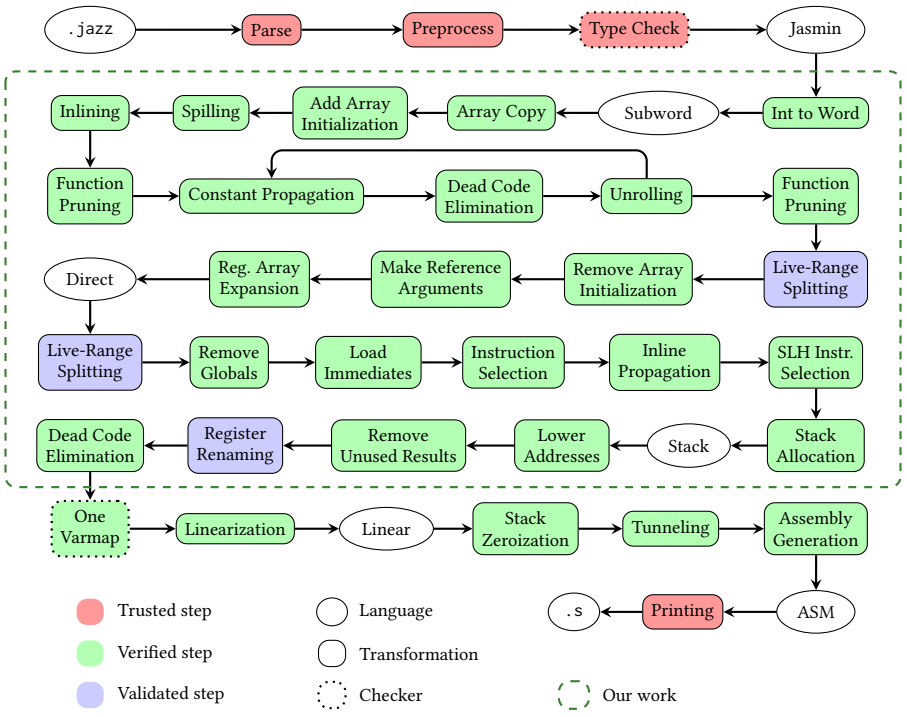


Fig. 6. Jasmin compiler intermediate languages and passes.

then a verified checker to ensure that the programs are alpha-equivalent. The checker for this pass is implemented and verified in `allocation.v` and `allocation_proof.v`.

(2) The Constant Propagation pass propagates the values of constants, simplifies complex expressions, and optimizes control flow constructs if their guard is constant. For example, the snippet $i := 3\,; x := i - 2$ maps to $i := 3\,; x := 1$. This pass is implemented and verified in `constant_prop.v` and `constant_prop_proof.v`.

(3) The Inlining pass replaces function calls to functions annotated by the programmer as `#inline` with their bodies. The pass renames the variables in the body of the callee to avoid clashes with those of the caller, reusing the infrastructure from Register Renaming. The transformation is performed in the order in which functions are defined, which means that we perform inlining once per function (as its callees have already been processed). This pass is implemented and verified in `inlining.v` and `inlining_proof.v`.

In the following section, we focus on the checker for Register Renaming, the constant branch optimization of Constant Propagation, and omit the renaming part of Inlining.

5.2 Correctness of the Jasmin Compiler

This section overviews our changes to the proofs of the Jasmin compiler and illustrates the use of different features of our logic presented in Section 4.3.

Compiler Correctness. The proof of each pass in the Jasmin front-end establishes an RHL program tuple of the form $=, = \models P \sim Q : \Phi_{\preccurlyeq} \Rightarrow \Psi_{\preccurlyeq}$, where P is the input program and Q its compilation, the handler contract is equality, and the relational specification $\Phi_{\preccurlyeq}$ and $\Psi_{\preccurlyeq}$ requires that each function and its compilation preserve the compiler simulation $\preccurlyeq$.

The simulation relation $\preccurlyeq$ involves a number of well-formedness conditions (e.g., that array values in the source correspond to pointers in the target) and relates values with the usual *more defined than* relation. Specifically, a target value is more defined than a source value if either they are equal or the latter is undefined. In the case of arrays, the simulation requires that a pointer in the target points to a region that is more defined than the corresponding array in the source. In the remainder of the section, we will omit this relation for simplicity and say that target and source values are equal.

The correctness of the Jasmin compiler front-end is states as follows. Note that the judgment is interpreted with equivalence up to source safety $\overset{u}{\approx}$, which means that the simulation relation holds until source errors occur.

Theorem 2 (Compiler Correctness). *Given a source program P and its compilation Q, we have*

$$=, = \models P \sim Q : \Phi_{\preccurlyeq} \Rightarrow \Psi_{\preccurlyeq}$$

under the conditions mentioned below.

Proof. As usual, the proof repeatedly applies the Trans rule to chain the proofs of each pass. It is formalized as `it_compiler_front_endP` in `proofs/compiler/it_compiler_proof.v`. □

The theorem relies on several assumptions about the initial states (e.g., that there is enough stack space in the target state) and about the arguments (e.g., that writable pointers are disjoint). These assumptions are similar to those in Theorem 4.

Register Renaming and Static Analyses. The checker in the Register Renaming pass checks that two programs are alpha-equivalent. It analyzes the code to compute a *renaming*, a map from source variables to target registers, at each program point. For example, the instruction $x := y + 3$ is alpha-equivalent to rdi := rdi + 3 if the initial renaming (before the assignment) is $\{y \mapsto \mathsf{rdi}\}$ and the resulting one (after the assignment) is $\{x \mapsto \mathsf{rdi}, y \mapsto \bot\}$.

The correctness of the checker states that the source and target states are equal up to renaming of register variables. The simulation relation is parameterized by a renaming $\alpha : \mathsf{Var} \rightharpoonup \mathsf{Var}$, which is a partial function from variables to variables. The simulation relation for this pass is as follows:

$$\sigma \preccurlyeq_\alpha \sigma' \triangleq \sigma_m = \sigma'_m \wedge \forall x \in \operatorname{dom} \alpha.\, \sigma(x) = \sigma'(\alpha(x)),$$

which means that memories should be identical, and if the renaming is defined on x, the variable maps coincide there.

Register Renaming is a natural fit for the simplified static analysis rules, since the pass itself is a checker; nevertheless,

For example, checking two assignments $\ell_1 := e_1$ and $\ell_2 := e_2$, succeeds only if there exists a renaming α such that e_2 is an renaming of e_1, and similarly for the left-values. Thus, the premises of the $\textsc{Assign}_{\mathsf{chk}}$ rule are trivially discharged. In this setting, the ($\textsc{Correct}_e$) condition requires that evaluating e_1 and e_2 in states equal up to α yields the same value, which we prove once and for all. Most passes of the compiler benefit from the simplified static analysis rules.

Constant Branch Elimination and One-Sided Rules. This transformation optimizes control flow constructs whose guards are constant—i.e., it removes constant branches. For example, the command **if** $(x = x)$ $\{c\}$ $\{c'\}$ maps to c, and **while** $(x < x)$ $\{c\}$ maps to **skip**. We prove it correct by showing that the source and target states are equal.

We use specialized one-sided rules deal with constant branches, as this transformation modifies the structure of the program. For instance, when a while loop has a false guard, we first use the $\textsc{Unroll}_L$ rule, which relates a while loop its one-step unrolling, and then the $\textsc{Case}_b$ to relate it to **skip**. All passes in Jasmin that modify the program structure use one-sided rules; further examples

are Instruction Selection to introduce assignments before complex computations and Dead Code Elimination to remove useless assignments.

Inlining and Different Semantics. This pass replaces function calls with the body of the callee if the latter is annotated as `#inline`. The proof for the Inlining pass is the most complex in the compiler after our changes, since a critical step in the argument requires a generic result on ITrees that is not provable in RHL directly. Proving the correctness of this pass is challenging because it requires to relate an ITree having $\mathsf{Call}(f, m, v)$ events with an ITree were parts of this event has already been interpreted as the semantics of the body of f on a state containing m and v. This is not expressible as one of our one-sided rules, as their equivalence holds only *after* interpreting Call events.

Our solution builds on a new handler for Call events, denoted H^I, to bring out the effects of inlining in a new *one-step inlining semantics*. Using this new semantics, we proceed in two steps: first, we show that the semantics of the source program is equivalent to its one-step inlining semantics; and secondly, we use RHL to prove that the one-step inlining semantics of the source program is equivalent to the semantics of the target program.

The new handler and semantics are as follows:

$$H^I(\mathsf{Call}(f,m,v)) \triangleq \begin{cases} H^{\mathrm{c}}(\mathsf{Call}(f,m,v)) & \text{if } f \text{ is } \texttt{\#inline}, \\ \mathsf{trigger}(\mathsf{Call}(f,m,v)) & \text{otherwise}, \end{cases} \qquad (\!|c|\!)^I_\sigma \triangleq \mathsf{interp}\big(H^I, (\!|c|\!)^{\mathrm{cer}}_\sigma\big).$$

This handler interprets a call to an `#inline` function f by executing its body using the usual call handler H^{c}, and other calls by re-triggering the original event. Based on this handler, the one-step inlining semantics interprets calls to `#inline` functions as ITrees that correspond to their bodies instead of Call events. Note that the bodies of `#inline` functions may themselves contain function calls, and these are not replaced, hence the name "one-step" inlining semantics.

Armed with this new semantics, we use existing lemmas from the ITree library that show that the interprocedural semantics of the source program (which interprets all Call events) is equivalent to interpreting Call events of its one-step inlining semantics. That is, for every command c and state σ,

$$(\!|c|\!)^{\mathrm{er}}_\sigma \approx \mathsf{interp_mrec}\big(H^{\mathrm{c}}, (\!|c|\!)^I_\sigma\big).$$

Intuitively, this result holds because both sides handle events in the same way, but the right-hand side handles some of them "earlier" with $(\!|\cdot|\!)^I$.

Finally, we use RHL to prove that the one-step inlining semantics of the source is equivalent to the semantics of the target. That is, for every source command c whose compilation is $\bar{c}$ and state σ,

$$\mathsf{interp_mrec}\big(H^{\mathrm{c}}, (\!|c|\!)^I_\sigma\big) \overset{u}{\approx} (\!|\bar{c}|\!)^{\mathrm{er}}_\sigma.$$

Notice that under the interp_mrec on either side, the left-hand side uses the one-step inlining semantics $(\!|\cdot|\!)^I$ and the right-hand the intraprocedural semantics $(\!|\cdot|\!)^{\mathrm{cer}}$. This is only possible with our generalization of the logic and of Theorem 1 to allow different semantics on the left- and right-hand sides. As mentioned in Section 4.3, our Rocq formalization implements this general version.

The general version of Theorem 1 is needed for three other passes besides Inlining, since there are four intermediate languages in the Jasmin compiler front-end. These other passes fit seamlessly the statement of the theorem and require no intermediate semantics.

Trade-Offs. We identify two potential improvements in our formalization of RHL, both of which relate to the checker abstractions. These improvements have limited use in the current Jasmin infrastructure and, thus, have not been implemented. However, we believe that they present no fundamental difficulty and could be of interest outside Jasmin.

First, the Stack Allocation pass would benefit for a more general type for $\mathcal{S}$ that involves states as well as values. This is because it places local variables—including arrays—in the stack frame of their function. For example, it transforms $a[1] := 1; x := a[1]$ into $[\mathrm{rsp} + 8] := 1; x := [\mathrm{rsp} + 8]$, where the array a now resides in memory, starting at address rsp. The simulation relation is more subtle than for previous passes, as we need to relate a source-level expression (e.g., $a[1]$) with a target-level machine word (e.g., rsp + 8) if the latter points to a memory region containing the value of the former. Thus, this pass would benefit from a more general type for $\mathcal{S}$ that takes the state into account, rather than just the value of the expressions—naturally, we can define $\mathcal{S}$ relative to a memory and fix the memory outside the RHL tuple, but this quickly becomes tedious.

Secondly, the SLH Instruction Selection pass would benefit from improved support of flow-sensitive analyses. This pass replaces selective speculative load hardening (SLH) operators with architecture-specific instructions. Selective speculative load hardening is a mitigation for Spectre attacks that has been recently adopted in Jasmin [11, 53] to protect against Spectre-v1 [39] and Spectre-RSB [40, 46]. The specifics of SLH are not relevant for our discussion; rather, what sets this pass apart is that it is the only one in the compiler that relies on a precise flow-sensitive static analysis. As a result, the simplified rules for control flow instructions such as $\textsc{Case}_{\mathsf{chk}}$ and its counterpart for loops cannot be used and we resort to the underlying general rules Case and While.

5.3 Comparison with Previous Work

Before our changes, the meaning of Jasmin programs was given by a big-step operational semantics of the form $\langle c, \sigma\rangle \Downarrow \sigma'$, stating that running a command c on a state σ terminates without errors and reaches a state σ'. The semantics of random sampling commands was given by an infinite random tape. Compiler correctness was stated as a forward simulation—that was later flipped backward—between a source command c and its compilation $\bar{c}$, stating that for every pair of source states σ and σ' and target state τ,

$$\langle c, \sigma\rangle \Downarrow \sigma' \wedge \sigma \preccurlyeq \tau \implies \exists \tau'. \langle \bar{c}, \tau\rangle \Downarrow \tau' \wedge \sigma' \preccurlyeq \tau',$$

where $\preccurlyeq$ is the simulation relation presented in Section 5.2 with an additional clause ensuring equal random tapes.

Consequently, the compiler correctness statement above ensured that safe and terminating Jasmin programs were compiled to safe and terminating programs with equal results. However, it did not give any guarantee on executions that diverge or on the probabilistic behavior (i.e., as distributions) of programs.

5.4 Reduced Proof Effort

The proof effort in the Jasmin compiler has been reduced as a result of our work; as a rough estimate, the size of the compiler front-end proofs has decreased by 10%. All in all, the Jasmin framework comprises 110K lines of Rocq and 31.7K lines of OCaml code. The Rocq part includes the compiler passes, their proofs, and the proof of generic assumptions on the architectures. The compiler back-end, not yet adapted to the ITree semantics, consists of five passes and amounts to 13K lines of Rocq.

Table 1 shows some statistics on the proof effort before and after the introduction of RHL. We show the instance of the Instruction Selection pass for each supported architecture separately. The Live-Range Splitting pass is absent because it is the composition of Register Renaming and Dead Code Elimination, and therefore already accounted for. Similarly, we omit Remove Unused Results since it is implemented inside Dead Code Elimination.

Table 1. Reduction in proof effort of the Jasmin compiler. The "Shared" column is the size—in lines of code (LOC)—of the auxiliary lemmas and definitions shared by both the previous and new proofs. The "Previous" and "New" columns present the size of preexisting and new proofs, respectively. The "Reduction" column is the reduction percentage including the "Shared" parts not modified by our work (i.e., one minus "New" plus "Shared" over "Previous" plus "Shared").

Compiler pass	Shared (LOC)	Previous (LOC)	New (LOC)	Reduction (%)
Int to Word	214	195	84	27.1
Array Copy	337	214	107	19.4
Array Initialization	352	263	137	20.5
Spilling	416	266	142	18.2
Inlining	255	339	283	9.4
Function Pruning	190	224	86	33.3
Constant Propagation	969	341	277	4.9
Dead Code Elimination	288	361	177	28.4
Unrolling	46	168	71	45.3
Remove Array Initialization	431	232	89	21.6
Reference Arguments	439	251	132	17.2
Register Array Expansion	674	243	219	2.6
Remove Globals	490	330	188	17.3
Load Immediates	142	277	100	42.2
x86-64 Instruction Selection	1681	254	90	8.5
ARMv7 Instruction Selection	1558	387	110	14.2
RISC-V Instruction Selection	583	173	72	13.4
SLH Instruction Selection	795	400	296	8.7
Inline Propagation	538	218	142	10.1
Stack Allocation	10 688	845	695	1.3
Lower Addresses	202	225	84	33.0
Register Renaming	433	445	198	28.1
Total	21 721	6651	3779	10.1

6 Preservation of Cryptographic Security

This section formalizes and proves preservation of cryptographic security. For concreteness, we focus on a single class of primitives, *key encapsulation mechanisms* (KEMs), and a single security property, *indistinguishability under chosen-ciphertext attacks* (IND-CCA). We show that the Jasmin compiler preserves IND-CCA for KEMs.

Preservation of cryptographic security is especially interesting for Jasmin, as a number of cryptographic primitives have been implemented in Jasmin and their security formally verified in EasyCrypt. At the same time, cryptographic security is particularly challenging to formalize and reason about, as it requires an interactive, probabilistic, almost surely terminating setting.

6.1 Cryptographic Background

A KEM is a public-key cryptographic primitive which allows two parties to share a secret key. Formally, KEMs are specified by three probabilistic algorithms: GenKey : $() \to \mathcal{PK} \times \mathcal{SK}$ to generate a public and a secret key, Encap : $\mathcal{PK} \to \mathcal{M} \times \mathcal{C}$ to generate a message and its encapsulation given a public key, and Decap : $\mathcal{SK} \times \mathcal{C} \to \mathcal{M}$ to decapsulate a message given a secret key, where $\mathcal{PK}$, $\mathcal{SK}$, $\mathcal{C}$, and $\mathcal{M}$ denote the sets of public keys, secret keys, encapsulations, and messages, respectively. Note that, in our setting, decapsulation never fails; this corresponds to the case of

$$
\begin{array}{l}
\mathrm{Exp}_{\mathscr{C},\mathscr{A}} \triangleq \\
\quad pk, sk \leftarrow \mathrm{GenKey}() \\
\quad \mathscr{A}_1{}^{\mathrm{Decap}(sk,\cdot)}(pk) \\
\quad c, m_0 \leftarrow \mathrm{Encap}(pk) \\
\quad m_1 \leftarrow \mathcal{U}(\mathcal{M}) \\
\quad b \leftarrow \mathcal{U}(\{0,1\}) \\
\quad g \leftarrow \mathscr{A}_2{}^{\mathrm{Decap}^c(sk,\cdot)}(c, m_b) \\
\quad \text{return } (g = b)
\end{array}
$$

(a)

$$
\begin{aligned}
[\![t]\!]^0 &\triangleq \mathbb{0} \\
[\![\mathsf{Ret}(r)]\!]^{n+1} &\triangleq \mathbb{1}_r \\
[\![\mathsf{Tau}(t)]\!]^{n+1} &\triangleq [\![t]\!]^n \\
[\![\mathsf{Vis}(\mathsf{Rnd}(v), k)]\!]^{n+1} &\triangleq \lambda r.\, \frac{1}{2^{8v}} \sum_a [\![k(a)]\!]^n(r) \\
[\![t]\!] &\triangleq \lim_{n\to\infty} [\![t]\!]^n
\end{aligned}
$$

(b)

Fig. 7. (a) IND-CCA experiment for KEMs. (b) Probabilistic semantics for ITrees.

KEMs with implicit rejection, of which ML-KEM is an instance. Key encapsulation mechanisms are required to be correct, in the sense that decapsulating a ciphertext recovers the shared key (message) with high probability.

The security of KEMs, IND-CCA, is modeled by a probabilistic experiment shown in Figure 7a; the KEM experiment is a variant of the real-or-random probabilistic experiment for public-key encryption. The experiment involves two adversaries $\mathscr{A}_1 : \mathcal{PK} \to ()$ and $\mathscr{A}_2 : C \times \mathcal{M} \to \{0,1\}$. Both adversaries share a state and can query a decapsulation oracle. Note however that the second adversary only has access to $\mathrm{Decap}^c(sk,\cdot)$, which is similar to $\mathrm{Decap}(sk,\cdot)$ but disallows calling the oracle with the challenge ciphertext c.

In the first stage, the challenger generates a key pair (pk, sk) and reveals the public key to the adversary $\mathscr{A}_1$, which may perform decapsulation queries to $\mathrm{Decap}(sk,\cdot)$. Then, the challenger calls the encapsulation oracle to generate an encapsulation-message pair (m_0, c), and samples a random message m_1. The challenger then samples a bit b uniformly and gives (m_b, c) to the adversary $\mathscr{A}_2$, which has access to the decapsulation oracle $\mathrm{Decap}^c(sk,\cdot)$, and must guess the value of b. The adversary wins if they guess the bit b correctly. More formally, the *advantage* $\mathrm{Adv}_{\mathscr{C}}(\mathscr{A})$ of an adversary $\mathscr{A} = (\mathscr{A}_1, \mathscr{A}_2)$ against a challenger $\mathscr{C}$ is how different the adversary's guess is from guessing uniformly at random, i.e., $\mathrm{Adv}_{\mathscr{C}}(\mathscr{A}) \triangleq \left|\Pr\left[\mathrm{Exp}_{\mathscr{C},\mathscr{A}} = 1\right] - \frac{1}{2}\right|$.

6.2 Formalizing IND-CCA with Interaction Trees

There are three challenges in formalizing IND-CCA security: the oracle queries are interactive, the algorithms are probabilistic, and they are not certainly terminating. All three are readily addressed using ITrees, as discussed in the previous sections. Oracle queries are represented as Orac events, which are similar to Call events but only for the three KEM algorithm. Random sampling is represented as Rnd events, and nontermination follows from the coinductive nature of ITrees.

We model the three components of the challenger as functions that return in the itree Rnd monad. This choice allows us to derive a challenger from a Jasmin program that implements a KEM, each component corresponding to the semantics of the respective function in the program, as we discuss in the next section.

Similarly, the components of an adversary return in the itree (Orac +′ Rnd) monad, as follows:

$$\mathscr{A}_1 : \mathcal{PK} \to \mathsf{itree}\ (\mathsf{Orac} +' \mathsf{Rnd})\ \mathcal{S}, \qquad \mathscr{A}_2 : \mathcal{S} \times C \times \mathcal{M} \to \mathsf{itree}\ (\mathsf{Orac} +' \mathsf{Rnd})\ \{0,1\}.$$

We model the internal state of the adversary explicitly as $\mathcal{S}$.

Finally, the IND-CCA experiment combines these parts in a tree of type itree (Orac +′ Rnd) $\{0,1\}$ using the monadic structure of ITrees. Since oracle queries from the adversary are Orac events, we deal with them defining a handler that uses Decap.

To define the advantage of an adversary we need to interpret ITrees as distributions: given a experiment E : itree Rnd $\{0,1\}$, we need to compute the probability that the output of the experiment is one. We do so generically, defining a probabilistic semantics $[\![\cdot]\!]$ for ITrees that will allow us to define the adversary's advantage as $\left|\Pr[[\![E]\!] = 1] - \frac{1}{2}\right|$.

Figure 7b presents the probabilistic interpretation of ITrees. Given a tree t : itree Rnd R, we first define its n-step approximation semantics $[\![t]\!]^n : \mathcal{D}(\mathsf{R})$, and then take the limit as n tends to infinity (we write $\mathcal{D}(\mathsf{R})$ for distributions over R). All zero-step approximations are the null distribution $\mathbb{0}$, which assigns zero probability to all results. All positive approximations of $\mathsf{Ret}(r)$ are the unit distribution on the result r, denoted $\mathbb{1}_r$, which assigns probability one to r. The $(n+1)$-step approximation of $\mathsf{Tau}(t)$ is the n-step approximation of t—note that this means that, as expected, diverging executions have probability zero. Finally, the $(n+1)$-step approximation of $\mathsf{Vis}(\mathsf{Rnd}(v), k)$ accumulates the probabilities over all possible random samples a and divides by the number of such possible samples—there are 2^{8v} vectors of v bytes. Finally, the probabilistic interpretation of t is denoted $[\![t]\!]$ and defined as the limit over n of $[\![t]\!]^n$ (which exists because $[\![t]\!]^n$ is increasing on n).

This semantics respects weak bisimilarity in the following sense.

Theorem 3. *Weakly bisimilar ITrees $t \approx t'$ give the same distribution, i.e., $[\![t]\!] = [\![t']\!]$.*

This theorem can be extended to allow trees that are weakly bisimilar w.r.t. a relation ϕ, as presented in Section B, which we formalize as `dinterp_eutt` in `proofs/compiler/distr.v`. Nevertheless, the simplified version stated above suffices to prove the preservation of IND-CCA.

6.3 Instantiation in Jasmin

To prove that Jasmin preserves IND-CCA security when compiling a program that implements a KEM, we first define the challengers induced by the source and compiled programs P and Q. We write `genkey`, `encap`, and `decap` for the names of the respective functions in the program, and `pk`, `sk`, `ct`, and `msg` for the names of the respective variables. We write σ for an initial state where no variable is defined. The challenger for P, denoted $\mathscr{C}_P$, is defined as follows:

$$\begin{aligned}
\mathrm{GenKey}() &\triangleq \sigma' \leftarrow (\!|P(\mathtt{genkey})|\!)^{\mathsf{er}}_{\sigma} ; \mathsf{Ret}(\sigma'(\mathtt{pk}), \sigma'(\mathtt{sk})) \\
\mathrm{Encap}(pk) &\triangleq \sigma' \leftarrow (\!|\mathtt{pk} \leftarrow pk|\!)^{\mathsf{LVal}}_{\sigma} ; \sigma'' \leftarrow (\!|P(\mathtt{encap})|\!)^{\mathsf{er}}_{\sigma'} ; \mathsf{Ret}(\sigma''(\mathtt{ct}), \sigma''(\mathtt{msg})) \\
\mathrm{Decap}(sk, c) &\triangleq \sigma' \leftarrow (\!|\mathtt{sk}, \mathtt{ct} \leftarrow sk, c|\!)^{\mathsf{LVal}}_{\sigma} ; \sigma'' \leftarrow (\!|P(\mathtt{decap})|\!)^{\mathsf{er}}_{\sigma'} ; \mathsf{Ret}(\sigma''(\mathtt{msg}))
\end{aligned}$$

The key generation algorithm simply executes the body of the `genkey` function in P and returns the values of `pk` and `sk` in the final state. The encapsulation algorithm first initializes `pk` with the given value pk, executes `encap`, and returns the values of `ct` and `msg`. Similarly, the decapsulation algorithm initializes `sk` and `ct` with the given values sk and c, executes `decap`, and returns the value of `msg`. Notice that the itree we give for GenKey(), Encap(pk) and Decap(sk, c) do not consider error events. This is because we consider in this section only safe programs, thus not error events can happen.

We omit the definition of the challenger for Q, denoted $\mathscr{C}_Q$, which is analogous except that instead of passing arguments as arrays in variables (e.g., `pk` in Encap), the variables contain pointers (64-bit words) to memory, and we write the values (e.g., pk in Encap) in memory. We write `p_pk`, `p_sk`, `p_ct`, and `p_msg` for the respective pointers.

Our formalization allows us to prove that Jasmin preserves IND-CCA security when compiling a KEM, as stated in the following theorem. The conditions are standard assumptions of compiler correctness, except the initialization part of the first condition. This is new and specific to the preservation of IND-CCA, and is necessary because—as most compilers—Jasmin provides no guarantees on result values that are uninitialized at source level.

Theorem 4 (The Jasmin Compiler Preserves Cryptographic Security). *Given a source program P with compilation Q and an adversary $\mathscr{A}$, we have that*

$$\mathrm{Adv}_{\mathscr{C}_P}(\mathscr{A}) = \mathrm{Adv}_{\mathscr{C}_Q}(\mathscr{A}),$$

under the following conditions:

(1) The source program P is safe and fully initializes its results.
(2) The initial target memory has enough allocated fresh memory for the stack of each function (the compiler statically determines how much stack space each function needs) and contains the data in the initial source memory.
(3) The pointer arguments `p_pk`, `p_sk`, `p_ct`, *and* `p_msg` *do not overflow, point to allocated fresh memory, and are pairwise disjoint.*

Proof. We show that the experiments for P and Q are weakly bisimilar, which means that their distributions coincide exactly (by Theorem 3). This proof if formalized as `reduction` in `proofs/compiler/end_to_end.v`. □

7 Related Work

We structure this section along the different lines of work related to our discussion.

Program Semantics. There is a long line of work that formalizes semantics for nonterminating and probabilistic computations in proof assistants. Coinductive program semantics and definitions of recursive functions are mechanized in [42, 47, 49]. Semantics of probabilistic programs are mechanized in [37] (discrete setting), [2, 36] (continuous setting), and [33] (tape-based). We refer the reader to [21] for further information about (mechanized and non-mechanized) approaches to semantics of probabilistic programs.

Verified Compilers. There is a long history of using proof assistants for proving functional correctness and for security properties of compilers; see [25]. Other works, e.g., [29], extend proofs assistants to ease compiler verification tasks. Here, we focus on key related works.

CompCert [43] is a compiler for C, written and proven correct Rocq. Early versions of CompCert used a big-step semantics; later versions are based on a small-step coinductive semantics. All proofs are based on simulation diagrams. Unlike CompCert, we benefit from ITrees and Relational Hoare Logic in our switch from big-step semantics to coinductive denotational semantics.

CakeML [41] is a compiler for Standard ML, written and verified in HOL. The semantics is defined in a functional big-step style with clocks to deal with divergence, and proofs are based on simulation diagrams and equational reasoning. Pancake [67] is an offspring of CakeML that uses an ITree semantics, proven equivalent to the functional big-step one, to reason about the interactions between programs and the external world.

Vellvm [66] is a compiler for the LLVM framework written and verified in Rocq. Vellvm uses the Single Static Assignment (SSA) representation and leverages SSA-specific proof techniques for proving correctness of program transformations. It uses ITrees to formalize LLVM-IR as a representation [64], to verify compiler optimizations and a Helix compiler to LLVM, and more recently relying on a two-phase memory model [23]. Unlike Jasmin, the semantics of LLVM is nondeterministic. Equivalence up-to-cutoff $\overset{u}{\approx}$ generalizes the specific extension of $\overset{e}{\approx}$ used in [23] to deal with out-of-memory errors in the target, which intuitively correspond to right cutoffs.

ProbCompCert [58] is an extension of CompCert for a fragment of the Stan language, supporting sampling from continuous distributions but no almost-sure termination. Zar [13] is a formally verified compiler from discrete probability programs to ITree samplers based on weakest preexpectations.

While the aforementioned works focus on the classic compiler correctness in terms of execution traces, stronger notions have also been considered. One such notion is robust compilation [1, 51], which asks that program behaviors are preserved under adversarial contexts. A main difference between robust compilation and our work is that it allows for source and target contexts to differ, so that the latter are able to capture low-level attacks. We plan to address low-level attacks in a separate (but compatible) effort, leveraging recent developments in hardware-software contracts [34]. Yet a different line of work studies preservation of information flow properties: Silver et al. [54] use the framework of ITrees to prove preservation of noninterference for an Imp-to-Asm compiler. Barthe et al. [15, 18, 19] prove preservation of constant-time for a core imperative language, CompCert, and an older version of the Jasmin compiler—the proof is not maintained but it could be of interest to restore it in the future. Constant-time has also been considered in the context of nondeterminism [26] and speculative side-channels, e.g., [10, 30].

(Relational) Programs Logics. Relational program logics provide a general framework for reasoning about 2-properties, i.e., properties of two programs. Relational Hoare Logic [24] is a program logic for reasoning about executions of two programs or two executions of the same program. Relational Separation Logic (RSL) [63] is a program logic for reasoning about execution of two heap-manipulating programs; it was developed concurrently with RHL. Iris [38] is a mechanized separation logic formalized in the Rocq prover, with several extensions that support relational reasoning. One of the primary applications of relational program logics is compiler correctness; for instance, Iris has been used to prove correctness of program transformations [31]. Recently, RSL has been used to prove correctness of (an idealization of) the tail-modulo constructor optimization used by the OCaml compiler [3].

Several works specifically develop program logics and related formalisms on top of ITrees. Vistrup et al. [61] develop a modular method for defining program logics within the Iris framework, relying on a generic definition of weakest precondition for ITrees, alongside effect libraries that can be combined together to deal with complex languages. Silver and Zdancewic [56] formalize the Dijkstra monad for weakest precondition reasoning with ITrees. Silver et al. [55] formalize a refinement calculus of top of ITrees. Finally, Michelland et al. [48] develop formally verified abstract interpreters using ITrees.

Independently, it would be interesting to explore extensions of our formalism to reason about optimizations of probabilistic programs, as, e.g., [45]. In this context, it would also be of interest to lift our relational logic to probabilistic programs, in the style of [12, 20].

Cryptographic Security. There is a large body of work that formalizes cryptographic security in proof assistants [14]. In particular, the EasyCrypt proof assistant [16, 17] has been used to mechanize the security proof of many constructions, including MLKEM [4]. EasyCrypt is based on pRHL (probabilistic Relational Hoare Logic) [20]. Other mechanized frameworks for cryptographic security include FCF [52], CryptHOL [22], SSProve [35], and IDPL [32]. Some works are more specifically focused on formalizing oracle systems. For instance, [60] presents a formalization of oracle systems in Lean. Their formalization has been further extended in [28] to reason about computational soundness.

Compiler Preservation of Cryptographic Security. Almeida et al. [6, Theorem 2] present a general method for proving preservation of cryptographic security against adversaries with acccess to side-channel leakage. However, their proof is carried on pen-and-paper, at a high level of abstraction, and elides many of the important details addressed by our work. We discuss side-channels in the context of our work in the conclusion.

8 Conclusion and Future Work

This paper proves that the Jasmin compiler front-end preserves IND-CCA security of KEMs. Our Rocq proof lays the foundations to be "combined" with the existing proofs of functional correctness and cryptographic security of the optimized Jasmin implementation of the post-quantum standard ML-KEM to conclude that the compiled code is also cryptographically secure. Note that the combination is necessarily informal (hence the quotes), as both proofs of functional correctness [8] and security [4] are carried using the EasyCrypt proof assistant [16, 17].

Our work assumes that the adversary models for source and target levels are equivalent. It would be possible to accomodate stronger target-level adversaries using techniques from hardware-software contracts [34], in particular to consider adversaries with access to side-channel leakage. On the programming language side, it will require proving that target programs are protected against side-channels. This can be done in two different ways: either by proving that the compiler also preserves protections againt side-channels or by proving directly that the assembly program is protected against side-channels, e.g., using a type system in the spirit of [57]. In the first case, it would be interesting to explore the potential benefits of ITrees for secure compilation.

Acknowledgments

We are very grateful for the help of Yannick Zakowski and Li-yao Xia, which was instrumental in the proof of Inlining. Likewise, we are greatly thankful to Pierre-Yves Strub and Jean-Christophe Léchenet, who generously helped us with the formalization of the theorems about probability distributions and Stack Allocation, respectively. This research was supported by the *Deutsche Forschungsgemeinschaft* (DFG, German research Foundation) as part of the Excellence Strategy of the German Federal and State Governments – EXC 2092 CASA - 390781972; and by the *Agence Nationale de la Recherche* (French National Research Agency) as part of the France 2030 programme – ANR-22-PECY-0006.

References

[1] Carmine Abate, Roberto Blanco, Ştefan Ciobâcă, Adrien Durier, Deepak Garg, Catalin Hritcu, Marco Patrignani, Éric Tanter, and Jérémy Thibault. 2020. Trace-Relating Compiler Correctness and Secure Compilation. In *Programming Languages and Systems - 29th European Symposium on Programming, ESOP 2020, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2020, Dublin, Ireland, April 25-30, 2020, Proceedings (Lecture Notes in Computer Science, Vol. 12075)*, Peter Müller (Ed.). Springer, 1–28. doi:10.1007/978-3-030-44914-8_1

[2] Reynald Affeldt, Cyril Cohen, and Ayumu Saito. 2023. Semantics of Probabilistic Programs using s-Finite Kernels in Coq. In *Proceedings of the 12th ACM SIGPLAN International Conference on Certified Programs and Proofs, CPP 2023, Boston, MA, USA, January 16-17, 2023*, Robbert Krebbers, Dmitriy Traytel, Brigitte Pientka, and Steve Zdancewic (Eds.). ACM, 3–16. doi:10.1145/3573105.3575691

[3] Clément Allain, Frédéric Bour, Basile Clément, François Pottier, and Gabriel Scherer. 2025. Tail Modulo Cons, OCaml, and Relational Separation Logic. *Proc. ACM Program. Lang.* 9, POPL (2025), 2337–2363. doi:10.1145/3704915

[4] José Bacelar Almeida, Santiago Arranz-Olmos, Manuel Barbosa, Gilles Barthe, François Dupressoir, Benjamin Grégoire, Vincent Laporte, Jean-Christophe Léchenet, Cameron Low, Tiago Oliveira, Hugo Pacheco, Miguel Quaresma, Peter Schwabe, and Pierre-Yves Strub. 2024. Formally Verifying Kyber - Episode V: Machine-Checked IND-CCA Security and Correctness of ML-KEM in EasyCrypt. In *Advances in Cryptology - CRYPTO 2024 - 44th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18-22, 2024, Proceedings, Part II (Lecture Notes in Computer Science, Vol. 14921)*, Leonid Reyzin and Douglas Stebila (Eds.). Springer, 384–421. doi:10.1007/978-3-031-68379-4_12

[5] José Bacelar Almeida, Manuel Barbosa, Gilles Barthe, Arthur Blot, Benjamin Grégoire, Vincent Laporte, Tiago Oliveira, Hugo Pacheco, Benedikt Schmidt, and Pierre-Yves Strub. 2017. Jasmin: High-Assurance and High-Speed Cryptography. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security, CCS 2017, Dallas, TX, USA, October 30 - November 03, 2017*, Bhavani Thuraisingham, David Evans, Tal Malkin, and Dongyan Xu (Eds.). ACM, 1807–1823. doi:10.1145/3133956.3134078

[6] José Bacelar Almeida, Manuel Barbosa, Gilles Barthe, and François Dupressoir. 2016. Verifiable Side-Channel Security of Cryptographic Implementations: Constant-Time MEE-CBC. In *Fast Software Encryption - 23rd International Conference,*

FSE 2016, Bochum, Germany, March 20-23, 2016, Revised Selected Papers (Lecture Notes in Computer Science, Vol. 9783), Thomas Peyrin (Ed.). Springer, 163–184. doi:10.1007/978-3-662-52993-5_9

[7] José Bacelar Almeida, Manuel Barbosa, Gilles Barthe, Benjamin Grégoire, Adrien Koutsos, Vincent Laporte, Tiago Oliveira, and Pierre-Yves Strub. 2020. The Last Mile: High-Assurance and High-Speed Cryptographic Implementations. In *2020 IEEE Symposium on Security and Privacy, SP 2020, San Francisco, CA, USA, May 18-21, 2020*. IEEE, 965–982. doi:10.1109/SP40000.2020.00028

[8] José Bacelar Almeida, Manuel Barbosa, Gilles Barthe, Benjamin Grégoire, Vincent Laporte, Jean-Christophe Léchenet, Tiago Oliveira, Hugo Pacheco, Miguel Quaresma, Peter Schwabe, Antoine Séré, and Pierre-Yves Strub. 2023. Formally verifying Kyber Episode IV: Implementation correctness. *IACR Trans. Cryptogr. Hardw. Embed. Syst.* 2023, 3 (2023), 164–193. doi:10.46586/TCHES.V2023.I3.164-193

[9] Krzysztof R. Apt, Frank S. de Boer, and Ernst-Rüdiger Olderog. 2009. *Verification of Sequential and Concurrent Programs*. Springer. doi:10.1007/978-1-84882-745-5

[10] Santiago Arranz-Olmos, Gilles Barthe, Lionel Blatter, Benjamin Grégoire, and Vincent Laporte. 2025. Preservation of Speculative Constant-Time by Compilation. *Proc. ACM Program. Lang.* 9, POPL (2025), 1293–1325. doi:10.1145/3704880

[11] Santiago Arranz-Olmos, Gilles Barthe, Chitchanok Chuengsatiansup, Benjamin Grégoire, Vincent Laporte, Tiago Oliveira, Peter Schwabe, Yuval Yarom, and Zhiyuan Zhang. 2025. Protecting Cryptographic Code Against Spectre-RSB: (and, in Fact, All Known Spectre Variants). In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2, ASPLOS 2025, Rotterdam, Netherlands, 30 March 2025 - 3 April 2025*, Lieven Eeckhout, Georgios Smaragdakis, Katai Liang, Adrian Sampson, Martha A. Kim, and Christopher J. Rossbach (Eds.). ACM, 933–948. doi:10.1145/3676641.3716015

[12] Martin Avanzini, Gilles Barthe, Davide Davoli, and Benjamin Grégoire. 2025. A Quantitative Probabilistic Relational Hoare Logic. *Proc. ACM Program. Lang.* 9, POPL (2025), 1167–1195. doi:10.1145/3704876

[13] Alexander Bagnall, Gordon Stewart, and Anindya Banerjee. 2023. Formally Verified Samplers from Probabilistic Programs with Loops and Conditioning. *Proc. ACM Program. Lang.* 7, PLDI (2023), 1–24. doi:10.1145/3591220

[14] Manuel Barbosa, Gilles Barthe, Karthik Bhargavan, Bruno Blanchet, Cas Cremers, Kevin Liao, and Bryan Parno. 2021. SoK: Computer-Aided Cryptography. In *42nd IEEE Symposium on Security and Privacy, SP 2021, San Francisco, CA, USA, 24-27 May 2021*. IEEE, 777–795. doi:10.1109/SP40001.2021.00008

[15] Gilles Barthe, Sandrine Blazy, Benjamin Grégoire, Rémi Hutin, Vincent Laporte, David Pichardie, and Alix Trieu. 2020. Formal verification of a constant-time preserving C compiler. *Proc. ACM Program. Lang.* 4, POPL (2020), 7:1–7:30. doi:10.1145/3371075

[16] Gilles Barthe, François Dupressoir, Benjamin Grégoire, César Kunz, Benedikt Schmidt, and Pierre-Yves Strub. 2013. EasyCrypt: A Tutorial. In *Foundations of Security Analysis and Design VII - FOSAD 2012/2013 Tutorial Lectures (Lecture Notes in Computer Science, Vol. 8604)*, Alessandro Aldini, Javier López, and Fabio Martinelli (Eds.). Springer, 146–166. doi:10.1007/978-3-319-10082-1_6

[17] Gilles Barthe, Benjamin Grégoire, Sylvain Heraud, and Santiago Zanella-Béguelin. 2011. Computer-Aided Security Proofs for the Working Cryptographer. In *Advances in Cryptology - CRYPTO 2011 - 31st Annual Cryptology Conference, Santa Barbara, CA, USA, August 14-18, 2011. Proceedings (Lecture Notes in Computer Science, Vol. 6841)*, Phillip Rogaway (Ed.). Springer, 71–90. doi:10.1007/978-3-642-22792-9_5

[18] Gilles Barthe, Benjamin Grégoire, and Vincent Laporte. 2018. Secure Compilation of Side-Channel Countermeasures: The Case of Cryptographic "Constant-Time". In *31st IEEE Computer Security Foundations Symposium, CSF 2018, Oxford, United Kingdom, July 9-12, 2018*. IEEE Computer Society, 328–343. doi:10.1109/CSF.2018.00031

[19] Gilles Barthe, Benjamin Grégoire, Vincent Laporte, and Swarn Priya. 2021. Structured Leakage and Applications to Cryptographic Constant-Time and Cost. In *CCS '21: 2021 ACM SIGSAC Conference on Computer and Communications Security, Virtual Event, Republic of Korea, November 15 - 19, 2021*, Yongdae Kim, Jong Kim, Giovanni Vigna, and Elaine Shi (Eds.). ACM, 462–476. doi:10.1145/3460120.3484761

[20] Gilles Barthe, Benjamin Grégoire, and Santiago Zanella-Béguelin. 2009. Formal certification of code-based cryptographic proofs. In *Proceedings of the 36th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2009, Savannah, GA, USA, January 21-23, 2009*, Zhong Shao and Benjamin C. Pierce (Eds.). ACM, 90–101. doi:10.1145/1480881.1480894

[21] Gilles Barthe, Joost-Pieter Katoen, and Alexandra Silva (Eds.). 2020. *Foundations of Probabilistic Programming*. Cambridge University Press. doi:10.1017/9781108770750

[22] David A. Basin, Andreas Lochbihler, and S. Reza Sefidgar. 2020. CryptHOL: Game-Based Proofs in Higher-Order Logic. *J. Cryptol.* 33, 2 (2020), 494–566. doi:10.1007/S00145-019-09341-Z

[23] Calvin Beck, Irene Yoon, Hanxi Chen, Yannick Zakowski, and Steve Zdancewic. 2024. A Two-Phase Infinite/Finite Low-Level Memory Model: Reconciling Integer-Pointer Casts, Finite Space, and undef at the LLVM IR Level of Abstraction. *Proc. ACM Program. Lang.* 8, ICFP (2024), 789–817. doi:10.1145/3674652

[24] Nick Benton. 2004. Simple relational correctness proofs for static analyses and program transformations. In *Proceedings of the 31st ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2004, Venice, Italy, January 14-16, 2004*, Neil D. Jones and Xavier Leroy (Eds.). ACM, 14–25. doi:10.1145/964001.964003

[25] J. Blanchette and A. Mahboubi (editors). 2026. *Handbook of Proof Assistants.* Birkhäuser.

[26] Owen Conoly, Andres Erbsen, and Adam Chlipala. 2025. Smooth, Integrated Proofs of Cryptographic Constant Time for Nondeterministic Programs and Compilers. *Proc. ACM Program. Lang.* 9, PLDI (2025), 1692–1715. doi:10.1145/3729318

[27] Jean Paul Degabriele, Kenneth G. Paterson, and Gaven J. Watson. 2011. Provable Security in the Real World. *IEEE Secur. Priv.* 9, 3 (2011), 33–41. doi:10.1109/MSP.2010.200

[28] Stefan Dziembowski, Grzegorz Fabianski, Daniele Micciancio, and Rafal Stefanski. 2025. Computationally-Sound Symbolic Cryptography in Lean. *IACR Cryptol. ePrint Arch.* (2025), 1700. https://eprint.iacr.org/2025/1700

[29] Oghenevwogaga Ebresafe, Ian Zhao, Ende Jin, Arthur Bright, Charles Jian, and Yizhou Zhang. 2025. Certified Compilers à la Carte. *Proc. ACM Program. Lang.* 9, PLDI (2025), 372–395. doi:10.1145/3729261

[30] Xaver Fabian, Marco Patrignani, Marco Guarnieri, and Michael Backes. 2025. Do You Even Lift? Strengthening Compiler Security Guarantees against Spectre Attacks. *Proc. ACM Program. Lang.* 9, POPL (2025), 893–922. doi:10.1145/3704867

[31] Lennard Gäher, Michael Sammler, Simon Spies, Ralf Jung, Hoang-Hai Dang, Robbert Krebbers, Jeehoon Kang, and Derek Dreyer. 2022. Simuliris: a separation logic framework for verifying concurrent program optimizations. *Proc. ACM Program. Lang.* 6, POPL (2022), 1–31. doi:10.1145/3498689

[32] Joshua Gancher, Kristina Sojakova, Xiong Fan, Elaine Shi, and Greg Morrisett. 2023. A Core Calculus for Equational Proofs of Cryptographic Protocols. *Proc. ACM Program. Lang.* 7, POPL (2023), 866–892. doi:10.1145/3571223

[33] Simon Oddershede Gregersen, Alejandro Aguirre, Philipp G. Haselwarter, Joseph Tassarotti, and Lars Birkedal. 2024. Asynchronous Probabilistic Couplings in Higher-Order Separation Logic. *Proc. ACM Program. Lang.* 8, POPL (2024), 753–784. doi:10.1145/3632868

[34] Marco Guarnieri, Boris Köpf, Jan Reineke, and Pepe Vila. 2021. Hardware-Software Contracts for Secure Speculation. In *42nd IEEE Symposium on Security and Privacy, SP 2021, San Francisco, CA, USA, 24-27 May 2021*. IEEE, 1868–1883. doi:10.1109/SP40001.2021.00036

[35] Philipp G. Haselwarter, Exequiel Rivas, Antoine Van Muylder, Théo Winterhalter, Carmine Abate, Nikolaj Sidorenco, Catalin Hritcu, Kenji Maillard, and Bas Spitters. 2023. SSProve: A Foundational Framework for Modular Cryptographic Proofs in Coq. *ACM Trans. Program. Lang. Syst.* 45, 3 (2023), 15:1–15:61. doi:10.1145/3594735

[36] Michikazu Hirata, Yasuhiko Minamide, and Tetsuya Sato. 2023. Semantic Foundations of Higher-Order Probabilistic Programs in Isabelle/HOL. In *14th International Conference on Interactive Theorem Proving, ITP 2023, July 31 to August 4, 2023, Białystok, Poland (LIPIcs, Vol. 268)*, Adam Naumowicz and René Thiemann (Eds.). Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 18:1–18:18. doi:10.4230/LIPICS.ITP.2023.18

[37] Joe Hurd, Annabelle McIver, and Carroll Morgan. 2004. Probabilistic Guarded Commands Mechanized in *HOL*. In *Proceedings of the Second Workshop on Quantitative Aspects of Programming Languages, QAPL 2004, Barcelona, Spain, March 27-28, 2004 (Electronic Notes in Theoretical Computer Science, Vol. 112)*, Antonio Cerone and Alessandra Di Pierro (Eds.). Elsevier, 95–111. doi:10.1016/J.ENTCS.2004.01.021

[38] Ralf Jung, David Swasey, Filip Sieczkowski, Kasper Svendsen, Aaron Turon, Lars Birkedal, and Derek Dreyer. 2015. Iris: Monoids and Invariants as an Orthogonal Basis for Concurrent Reasoning. In *Proceedings of the 42nd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2015, Mumbai, India, January 15-17, 2015*, Sriram K. Rajamani and David Walker (Eds.). ACM, 637–650. doi:10.1145/2676726.2676980

[39] Paul Kocher, Jann Horn, Anders Fogh, Daniel Genkin, Daniel Gruss, Werner Haas, Mike Hamburg, Moritz Lipp, Stefan Mangard, Thomas Prescher, Michael Schwarz, and Yuval Yarom. 2020. Spectre attacks: exploiting speculative execution. *Commun. ACM* 63, 7 (2020), 93–101. doi:10.1145/3399742

[40] Esmaeil Mohammadian Koruyeh, Khaled N. Khasawneh, Chengyu Song, and Nael B. Abu-Ghazaleh. 2024. Spectre Returns! Speculation Attacks Using the Return Stack Buffer. *IEEE Des. Test* 41, 2 (2024), 47–55. doi:10.1109/MDAT.2024.3352537

[41] Ramana Kumar, Magnus O. Myreen, Michael Norrish, and Scott Owens. 2014. CakeML: a verified implementation of ML. In *The 41st Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL '14, San Diego, CA, USA, January 20-21, 2014*, Suresh Jagannathan and Peter Sewell (Eds.). ACM, 179–192. doi:10.1145/2535838.2535841

[42] Xavier Leroy. 2006. Coinductive Big-Step Operational Semantics. In *Programming Languages and Systems, 15th European Symposium on Programming, ESOP 2006, Held as Part of the Joint European Conferences on Theory and Practice of Software, ETAPS 2006, Vienna, Austria, March 27-28, 2006, Proceedings (Lecture Notes in Computer Science, Vol. 3924)*, Peter Sestoft (Ed.). Springer, 54–68. doi:10.1007/11693024_5

[43] Xavier Leroy. 2006. Formal certification of a compiler back-end, or: programming a compiler with a proof assistant. In *33rd ACM symposium on Principles of Programming Languages*. ACM Press, 42–54.

[44] Xavier Leroy and Hervé Grall. 2008. Coinductive big-step operational semantics.

[45] Jianlin Li, Eric Wang, and Yizhou Zhang. 2024. Compiling Probabilistic Programs for Variable Elimination with Information Flow. *Proc. ACM Program. Lang.* 8, PLDI (2024), 1755–1780. doi:10.1145/3656448

[46] Giorgi Maisuradze and Christian Rossow. 2018. ret2spec: Speculative Execution Using Return Stack Buffers. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security, CCS 2018, Toronto, ON, Canada, October 15-19, 2018*, David Lie, Mohammad Mannan, Michael Backes, and XiaoFeng Wang (Eds.). ACM, 2109–2122. doi:10.1145/3243734.3243761

[47] Conor McBride. 2015. Turing-Completeness Totally Free. In *Mathematics of Program Construction - 12th International Conference, MPC 2015, Königswinter, Germany, June 29 - July 1, 2015. Proceedings (Lecture Notes in Computer Science, Vol. 9129)*, Ralf Hinze and Janis Voigtländer (Eds.). Springer, 257–275. doi:10.1007/978-3-319-19797-5_13

[48] Sébastien Michelland, Yannick Zakowski, and Laure Gonnord. 2024. Abstract Interpreters: A Monadic Approach to Modular Verification. *Proc. ACM Program. Lang.* 8, ICFP (2024), 28 pages. doi:10.1145/3674646

[49] Keiko Nakata and Tarmo Uustalu. 2009. Trace-Based Coinductive Operational Semantics for While. In *Theorem Proving in Higher Order Logics, 22nd International Conference, TPHOLs 2009, Munich, Germany, August 17-20, 2009. Proceedings (Lecture Notes in Computer Science, Vol. 5674)*, Stefan Berghofer, Tobias Nipkow, Christian Urban, and Makarius Wenzel (Eds.). Springer, 375–390. doi:10.1007/978-3-642-03359-9_26

[50] National Institute of Standards, Technology (NIST), Gorjan Alagic, Quynh Dang, Dustin Moody, Angela Robinson, Hamilton Silberg, and Daniel Smith-Tone. 2024. Module-Lattice-Based Key-Encapsulation Mechanism Standard. doi:10.6028/NIST.FIPS.203

[51] Marco Patrignani, Robert Künnemann, Riad S. Wahby, and Ethan Cecchetti. 2024. Universal Composability Is Robust Compilation. *ACM Trans. Program. Lang. Syst.* 46, 4 (2024), 13:1–13:64. doi:10.1145/3698234

[52] Adam Petcher and Greg Morrisett. 2015. The Foundational Cryptography Framework. In *Principles of Security and Trust - 4th International Conference, POST 2015, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2015, London, UK, April 11-18, 2015, Proceedings (Lecture Notes in Computer Science, Vol. 9036)*, Riccardo Focardi and Andrew C. Myers (Eds.). Springer, 53–72. doi:10.1007/978-3-662-46666-7_4

[53] Basavesh Ammanaghatta Shivakumar, Gilles Barthe, Benjamin Grégoire, Vincent Laporte, Tiago Oliveira, Swarn Priya, Peter Schwabe, and Lucas Tabary-Maujean. 2023. Typing High-Speed Cryptography against Spectre v1. In *44th IEEE Symposium on Security and Privacy, SP 2023, San Francisco, CA, USA, May 21-25, 2023*. IEEE, 1094–1111. doi:10.1109/SP46215.2023.10179418

[54] Lucas Silver, Paul He, Ethan Cecchetti, Andrew K. Hirsch, and Steve Zdancewic. 2023. Semantics for Noninterference with Interaction Trees. In *37th European Conference on Object-Oriented Programming, ECOOP 2023, July 17-21, 2023, Seattle, Washington, United States (LIPIcs, Vol. 263)*, Karim Ali and Guido Salvaneschi (Eds.). Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 29:1–29:29. doi:10.4230/LIPICS.ECOOP.2023.29

[55] Lucas Silver, Eddy Westbrook, Matthew Yacavone, and Ryan Scott. 2023. Interaction Tree Specifications: A Framework for Specifying Recursive, Effectful Computations That Supports Auto-Active Verification. In *37th European Conference on Object-Oriented Programming (ECOOP 2023) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 263)*, Karim Ali and Guido Salvaneschi (Eds.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 30:1–30:26. doi:10.4230/LIPIcs.ECOOP.2023.30

[56] Lucas Silver and Steve Zdancewic. 2021. Dijkstra monads forever: termination-sensitive specifications for interaction trees. *Proc. ACM Program. Lang.* 5, POPL (2021), 28 pages. doi:10.1145/3434307

[57] Shixin Song, Tingzhen Dong, Kosi Nwabueze, Julian Zanders, Andres Erbsen, Adam Chlipala, and Mengjia Yan. 2025. Securing Cryptographic Software via Typed Assembly Language (Extended Version). *CoRR* abs/2509.08727 (2025). arXiv:2509.08727 doi:10.48550/ARXIV.2509.08727

[58] Joseph Tassarotti and Jean-Baptiste Tristan. 2023. Verified Density Compilation for a Probabilistic Programming Language. *Proc. ACM Program. Lang.* 7, PLDI (2023), 615–637. doi:10.1145/3591245

[59] The Rocq Development Team. 2025. *Rocq Prover Reference Manual* (9.0.0 ed.). INRIA & CNRS. doi:10.5281/zenodo.15149629 Available at https://docs.rocq-prover.org/master/refman/ (accessed 2025-11-08).

[60] Devon Tuma and Nicholas Hopper. 2024. VCVio: A Formally Verified Forking Lemma and Fiat-Shamir Transform, via a Flexible and Expressive Oracle Representation. *IACR Cryptol. ePrint Arch.* (2024), 1819. https://eprint.iacr.org/2024/1819

[61] Max Vistrup, Michael Sammler, and Ralf Jung. 2025. Program Logics à la Carte. *Proc. ACM Program. Lang.* 9, POPL (2025), 32 pages. doi:10.1145/3704847

[62] Li-yao Xia, Yannick Zakowski, Paul He, Chung-Kil Hur, Gregory Malecha, Benjamin C. Pierce, and Steve Zdancewic. 2020. Interaction trees: representing recursive and impure programs in Coq. *Proc. ACM Program. Lang.* 4, POPL (2020), 51:1–51:32. doi:10.1145/3371119

[63] Hongseok Yang. 2007. Relational separation logic. *Theor. Comput. Sci.* 375, 1-3 (2007), 308–334. doi:10.1016/J.TCS.2006.12.036

[64] Yannick Zakowski, Calvin Beck, Irene Yoon, Ilia Zaichuk, Vadim Zaliva, and Steve Zdancewic. 2021. Modular, compositional, and executable formal semantics for LLVM IR. *Proc. ACM Program. Lang.* 5, ICFP (2021), 1–30.

Table 2. A summary of Jasmin compiler passes.

Compiler pass	Summary
Preprocess	Resolve syntactic sugar and imports.
Int to Word	Replace word-sized integers with machine words.
Array Copy	Expand array copies into `for` loops.
Add Array Initialization	Insert array initializations at the beginning of functions.
Spilling	Replace spill and unspill operators with copies to and from fresh variables.
Inlining	Replace (some) function calls with their bodies.
Function Pruning	Remove unused functions.
Constant Propagation	Propagate the values of variables and optimize expressions with constants.
Dead Code Elimination	Remove unnecessary assignments.
Unrolling	Unroll `for` loops.
Live-Range Splitting	Transform the program into SSA form.
Remove Array Initialization	Remove array initializations.
Reference Arguments	Use intermediate pointer variables instead of array arguments.
Register Array Expansion	Expand register arrays into variables.
Remove Globals	Move local global values to the top-level.
Load Immediates	Store immediates in temporary registers.
Instruction Selection	Replace assignments with architecture-specific operations.
Inline Propagation	Propagate the values of inline variables.
SLH Instruction Selection	Replace SLH operators with architecture-specific operations.
Stack Allocation	Replace stack variables with concrete stack positions.
Lower Addresses	Use only addresses allowed by the architecture.
Remove Unused Results	Remove unused results of functions.
Register Renaming	Rename all variables to use architecture-specific registers.
One Varmap	Verify that local variables can be lifted to a shared state.
Linearization	Replace structured control flow with jumps.
Stack Zeroization	Overwrite used stack memory at the end of exported functions.
Tunneling	Simplify chains of jumps.
Assembly Generation	Use concrete registers, flags, and conditions instead of variables.

doi:10.1145/3473572

[65] Yannick Zakowski, Paul He, Chung-Kil Hur, and Steve Zdancewic. 2020. An equational theory for weak bisimulation via generalized parameterized coinduction. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020)*. Association for Computing Machinery, 71–84. doi:10.1145/3372885.3373813

[66] Jianzhou Zhao, Santosh Nagarakatte, Milo M. K. Martin, and Steve Zdancewic. 2012. Formalizing the LLVM intermediate representation for verified program transformations. In *Proceedings of the 39th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2012, Philadelphia, Pennsylvania, USA, January 22-28, 2012*, John Field and Michael Hicks (Eds.). ACM, 427–440. doi:10.1145/2103656.2103709

[67] Junming Zhao, Miki Tanaka, Johannes Åman Pohjola, Alessandro Legnani, Tiana Tsang Ung, H. Truong, Tsun Wang Sau, Thomas Sewell, Rob Sison, Hira Syeda, Magnus Myreen, Michael Norrish, and Gernot Heiser. 2025. Verifying Device Drivers with Pancake.

A Summary of Jasmin Compiler Passes

Table 2 presents a one-sentence summary of each pass in the Jasmin compiler.

B Probabilistic Liftings

Lifting a relation between two sets X and Y to one between distributions over X and Y is necessary to reason about the probabilistic semantics of programs. In this section, we review the standard approach to achieve this lifting.

A crucial advantage of building the semantics of our language with ITrees—and using equivalence up-to-tau as our compiler correctness notion—is that Theorem 3 theorem is independent of the language and compiler, and that the compiler correctness theorem Theorem 2 is independent of the probabilistic semantics. Specifically, we can elide all probability details when proving the compiler correct, and, afterward and separately, use Theorem 3 to achieve strong guarantees about the probabilistic behavior of programs.

Unfortunately, the statement of Theorem 3 is too restrictive for most compilers, as the compiler correctness result is expressed relative to a simulation relation $\preccurlyeq$. We need to refine the theorem to account for such a relation, which means that we must relate the probability of returning a source state with that of returning a target state related by the simulation. Moreover, since simulation relations are seldom bijective, we need to further relax this statement to consider the probability of sets of states, where one is the image of the other under the simulation relation. Let us now reformulate the theorem with these distinctions in mind. Recall that the probability under a distribution μ of a subset X is $\Pr_\mu[X] \triangleq \sum_{x \in X} \mu(x)$. In our case, we are concerned with subsets of source and target states, for the source and target semantics, respectively.

Theorem 5. *Given two ITrees that are in simulation up-to-tau $t \approx_\preccurlyeq t'$, and a subset S of source states and a subset T of target states such that*

$$\forall \sigma \tau.\, \sigma \preccurlyeq \tau \implies (\sigma \in S \iff \tau \in T)$$

then

$$\Pr_{[\![t]\!]}[S] = \Pr_{[\![t']\!]}[T].$$